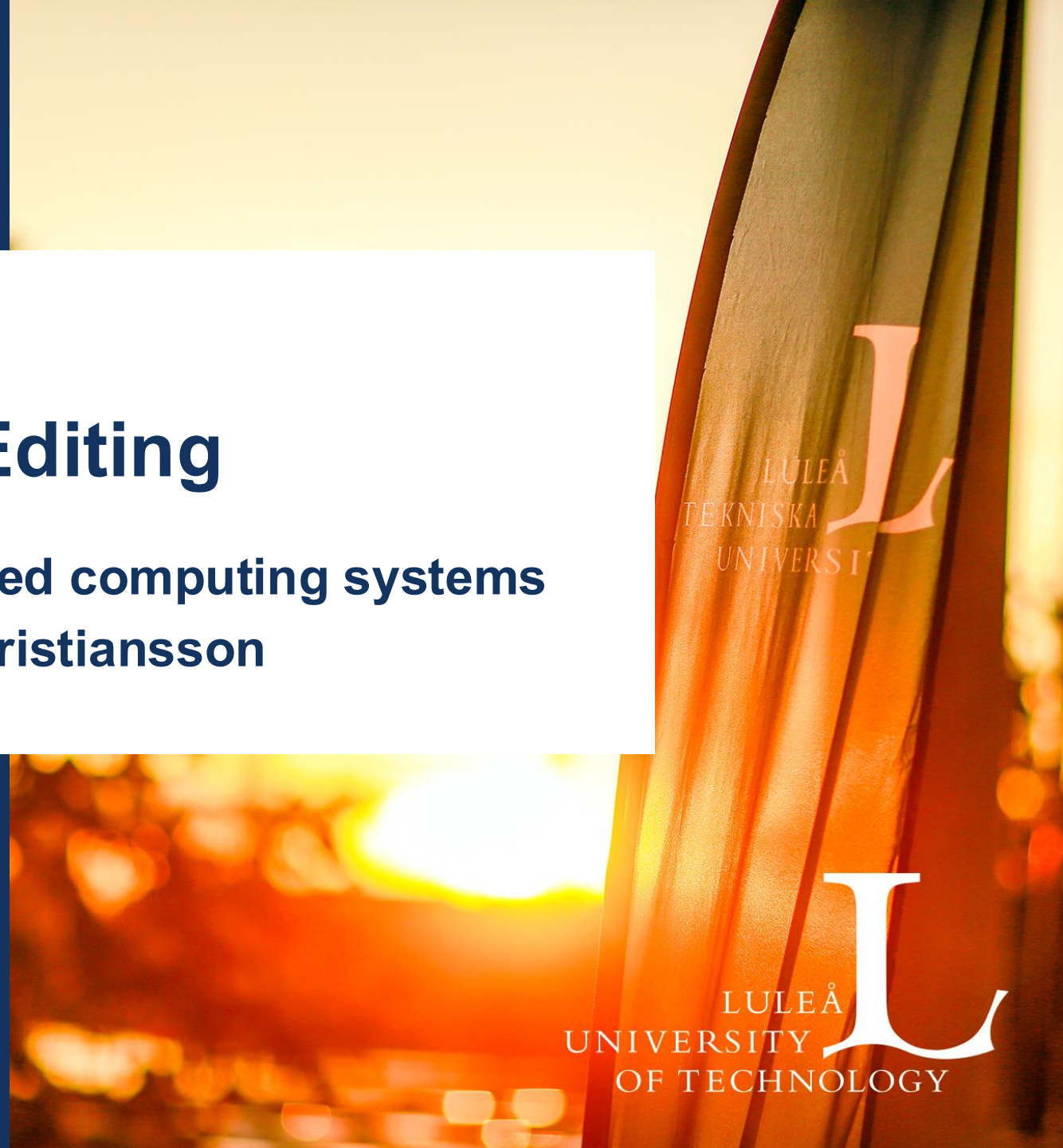
Collaborative Editing

Mobile and distributed computing systems

Johan Kristiansson



Trade-offs

CP Systems (Consistency + Partition Tolerance)

- **Trade-off:** Wait for consistency, system might be temporarily unavailable
- Correctness is important than availability (e.g. banking)
- **Example:** MongoDB, Etcd, Zookeeper

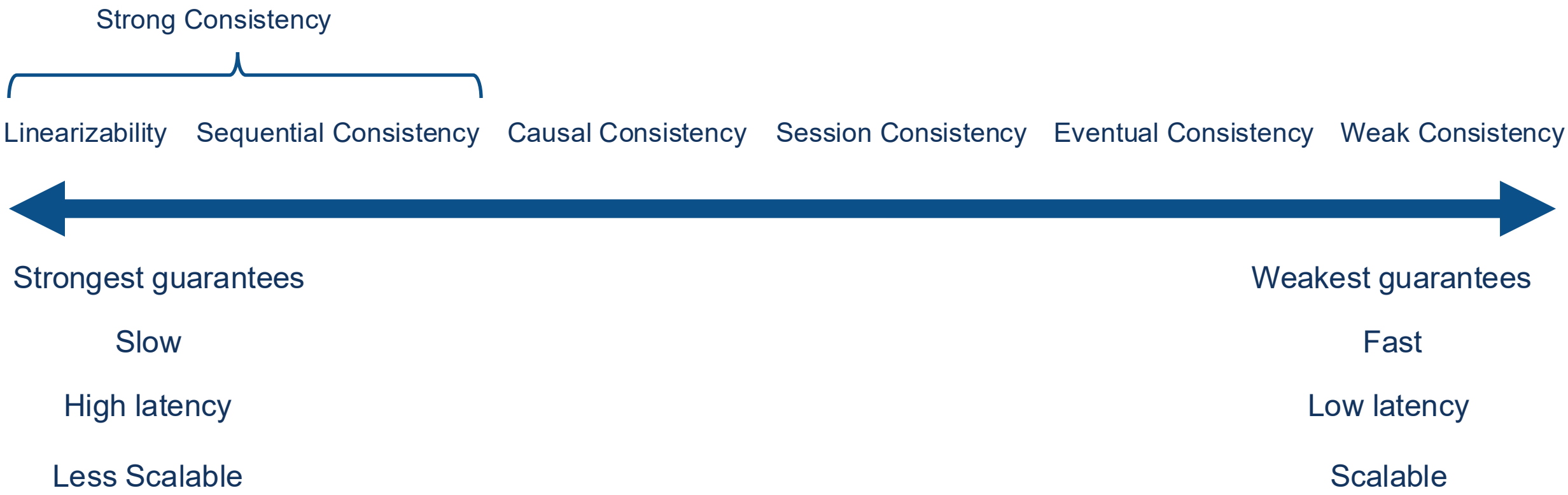
AP Systems (Availability + Partition Tolerance)

- **Trade-off:** Always available, system might be inconsistent
- Availability is more important than perfect consistency
- **Example**: Offline-first apps, Kademlia

CA Systems (Consistency + Availability)

- **Trade-off:** The only way to prevent network partitions is to use single-node systems (not distributed)
- **Example:** Single PostgreSQL instance

Consistency Models



Ordering

Total Order

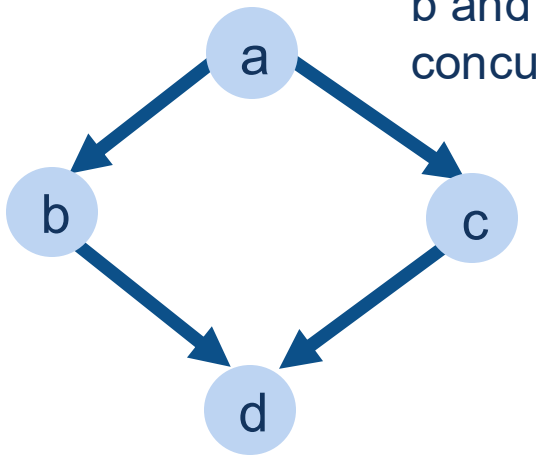
- Every pair of events is comparable
- One global sequence everyone agrees on



Total Order: a,b,c,d

Causal Order

- Events are ordered only if one causally influences another (i.e., happens-before)
- Allows more concurrency, less overhead



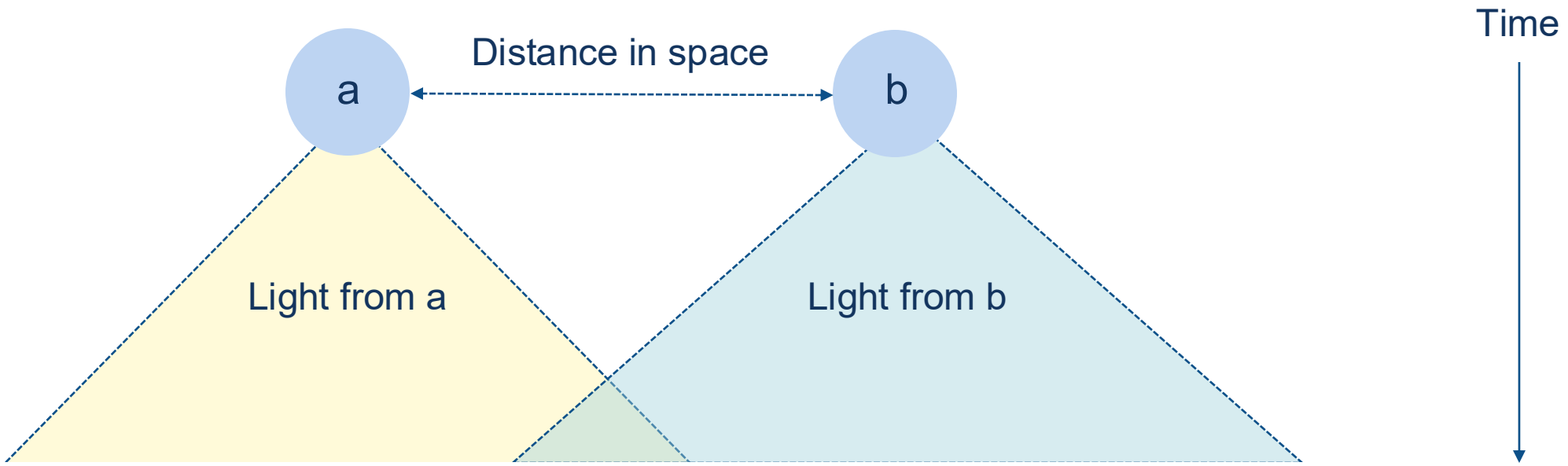
b and c happens concurrently

Valid Total Orders: a,b,c,d or a,c,b,d

$a \prec b$ means event a comes before event b in total order

$a \prec b \prec c \prec d$

Causality



Happened-before (denoted $\rightarrow$) encodes potential causuality
Relation between two events so that one event happened before the other event

When $a \rightarrow b$, then event **a might** have caused event **b**
When $a \parallel b$, then event **a cannot** have caused event **b**

Transitivity

$\forall a, b, c \quad a \rightarrow b \text{ and } b \rightarrow c \Rightarrow a \rightarrow c$

Asymmetry

$\forall a, b \quad a \rightarrow b \Rightarrow b \nrightarrow a$

Irreflexively

$\forall a, a \nrightarrow a$

Logical Clocks

- Mechanism to capture causal relationships in distributed systems
  - First Logical Clock was developed by Leslie Lamport in 1978 (Turing Award 2013)
- Lamport Timestamp
  - Assigns a numerical timestamp (L) to each event to preserve causal relationships
  - If $a \rightarrow b \Rightarrow L(a) < L(b)$ $\quad L(a) < L(b) \not\Rightarrow a \rightarrow b$

Since a and b could happen concurrently

Local Event

```
def on_local_event():
   local_clock += 1
   return local_clock
```

Sending

```
def send_message(msg):
    local_clock += 1
    msg.timestamp = local_clock send(msg)
```

Receiving

```
def receive_message(msg):
    local_clock = max(local_clock, msg.timestamp)+1
    process(msg)
```

Vector Clock

Extension of Lamport Timestamps that can detect concurrent events

- Preserves causality and identifies concurrent operations
- Each process p_i maintains vector $VC_i[1..n]$ where n = number of processes

Local Event

$$VC_i[i] = VC_i[i] + 1$$

Sending

$$VC_i[i] = VC_i[i] + 1$$

$$msg.VC = VC_i$$

Receiving

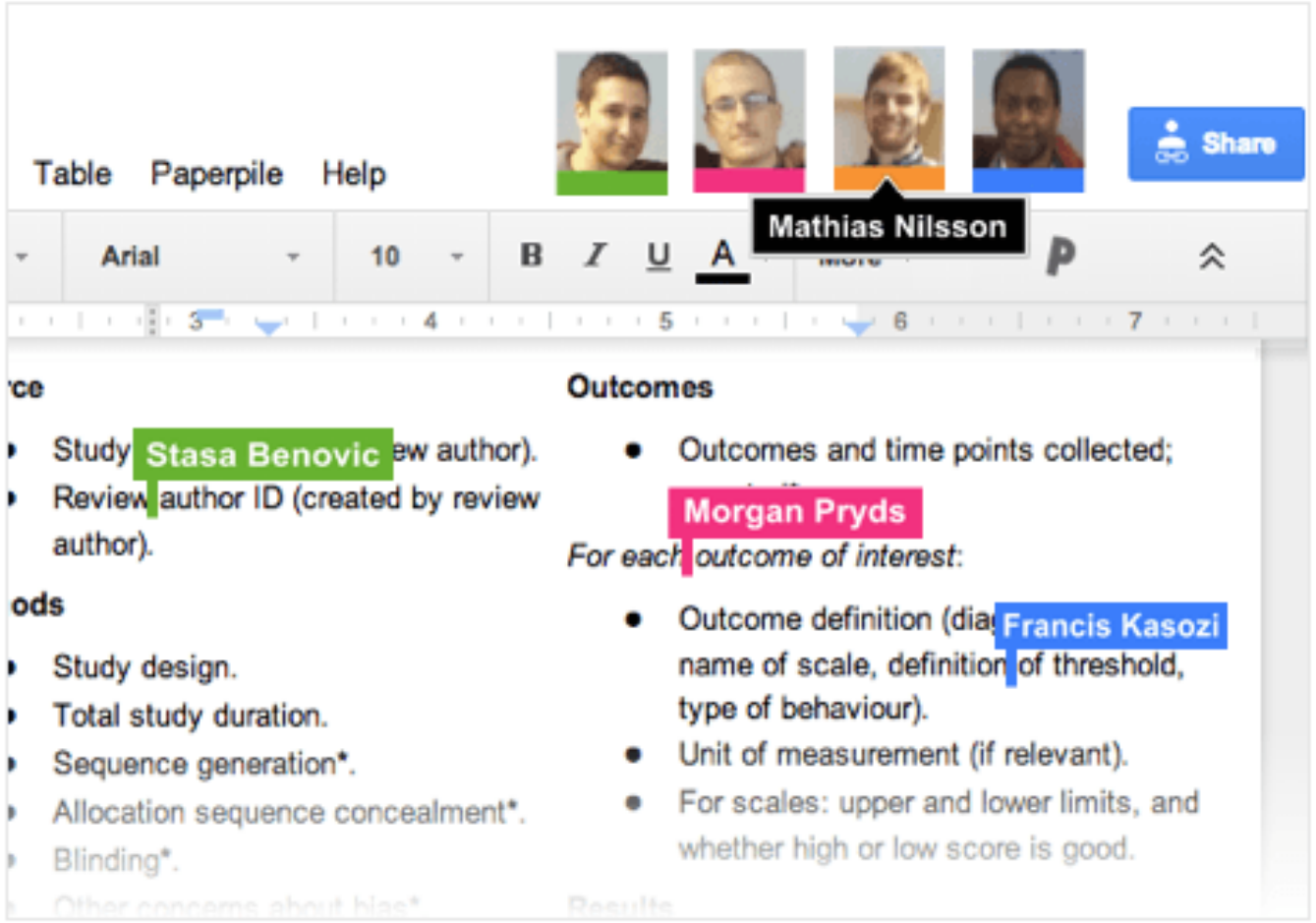
$$\forall k \in [1..n] : VC_k = max(VC_i[k], msg.VC[k])$$

$$VC_i[i] = VC_i[i] + 1$$

Collaborative Editing

Collaborative editing is the process of multiple people editing the same document simultaneously

- Everyone can see and make changes to the same document at the same time, often seeing each other's edits instantly
- Shared documents, Word, Google Docs, Online pair-programming/Collaborative IDE, shared whiteboards, collaborative design tools etc



https://paperpile.com/blog/google-docs-in-the-university/

Turn-based Editing

1. Request an edit document token from the server
2. Wait until the server says it's our turn to edit the document
3. Tell the server how to edit the document
4. Release the edit document token

Turn-based Editing

1. Request an edit document token from the server
2. Wait until the server says it's our turn to edit the document
3. Tell the server how to edit the document
4. Release the edit document token

This does not work very well, right?

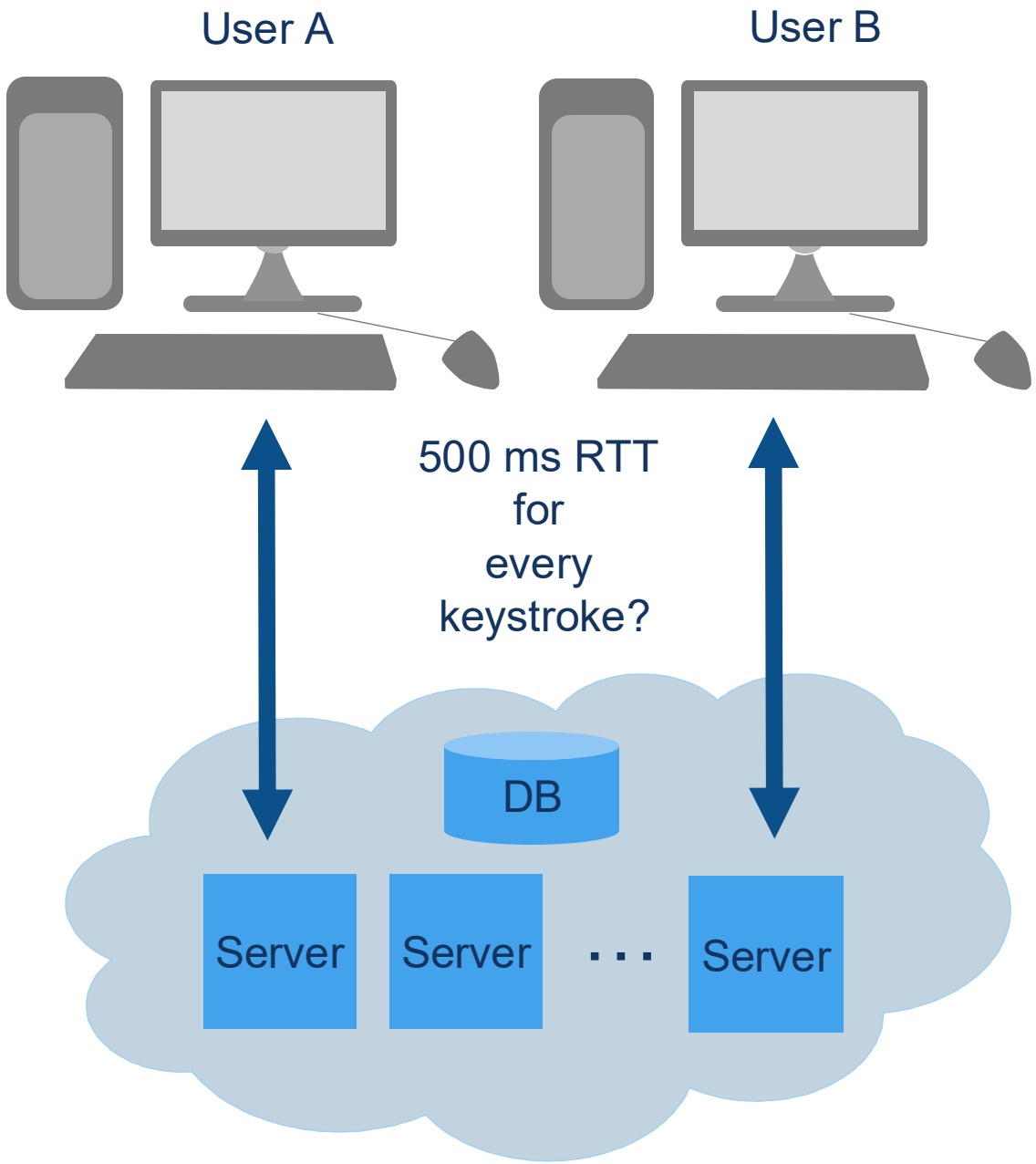
Lock-based solutions are too slow

Network latency kills user experience

- Every keystroke → Server roundtrip → 50-200ms delay
- Server processing: 10-50ms per operation
- Total per action: 50 - 400ms of waiting

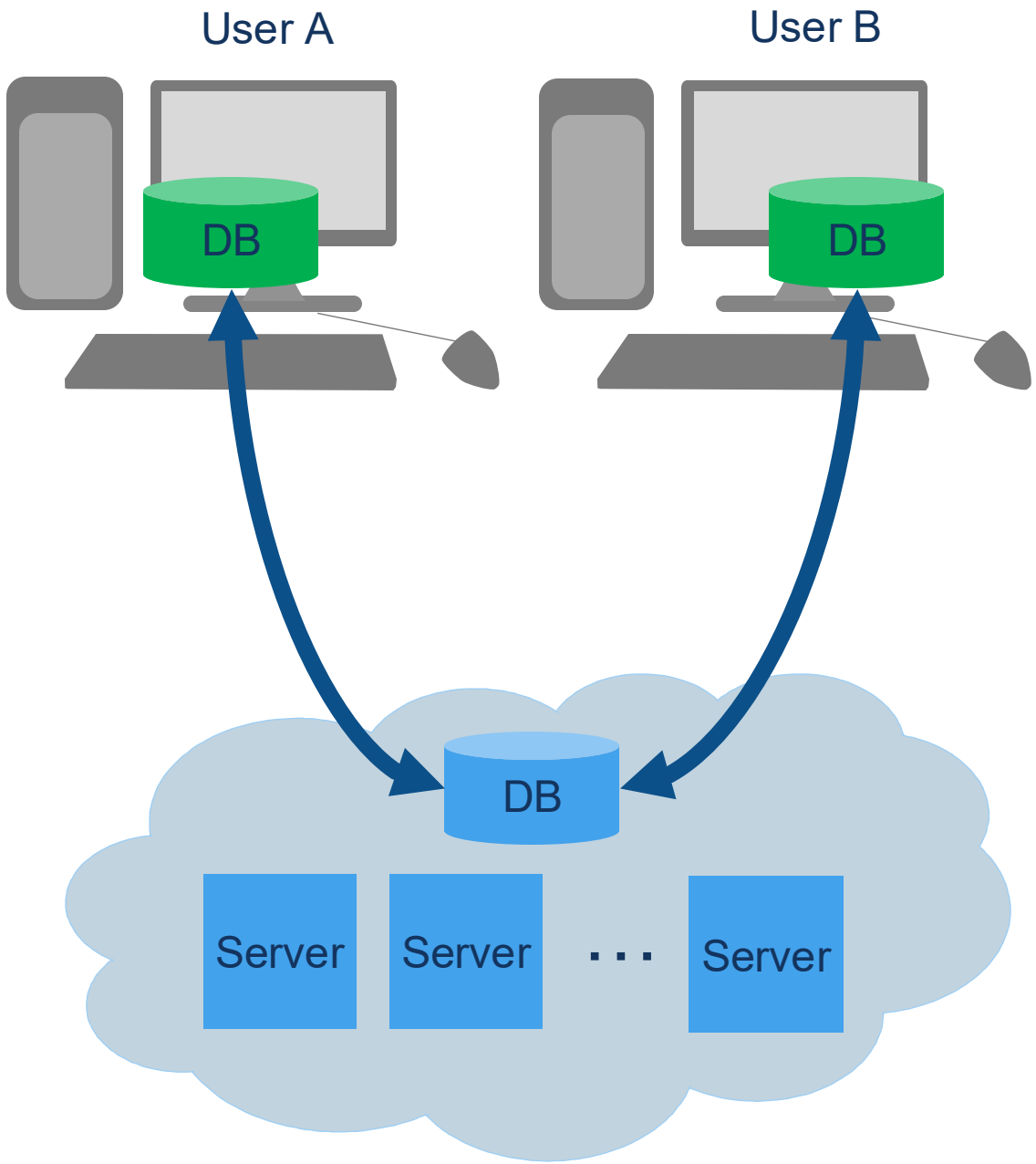
Why wait for server fails?

- Typing feels broken - Characters appear after noticeable lag
- Cursor jumps around - Position conflicts with server state
- Unresponsive UI - Click... wait... update
- Unusable on mobile networks (high latency)



Principles of Collaborative Editing

1. **Apply changes locally** (optimistic updates)
   Execute operations immediately on local data without waiting for server confirmation
2. **Sync with other users** (in the background)
   Propagate local changes to other users and receive their changes asynchronously
3. **Resolve Conflicts (**automatically)
   When concurrent edits happen, use algorithms to merge them without user intervention
4. **Preserve user intent**
   The system understands what the user was trying to do and maintains that goal even when the document changes.



Operational Transformation

A **concurrency control algorithm** used in **real-time collaborative editing** systems

Enables **multiple users** to edit the same document simultaneously **without conflicts**

Key Concepts

- **Operations:** Insert, delete, update actions on shared data.
- **Transformation:** Adjusts operations to maintain consistency across all user views.
- **Intention Preservation:** Ensures each user's intended changes are preserved, even when edits overlap

Insert and Delete Operations

Insert operation

$\text{ins}(p, c) = \text{insert character } c \text{ at position } p$

$\text{ins}(2, \text{X})$ applied to document abc yields new document abXc

Delete operation

$\text{del}(p) = \text{delete character at position } p$

$\text{del}(1)$ applied to document abc yields new document ac

Insert and Delete Operations

Insert operation

$\text{ins}(p, c) = \text{insert character } c \text{ at position } p$

$\text{ins}(2, \text{X})$ applied to document abc yields new document abXc

Delete operation

$\text{del}(p) = \text{delete character at position } p$

$\text{del}(1)$ applied to document abc yields new document ac

Example of other Rich Text Editor Operations

$\text{ins}(\text{pos, text}), \text{del}(\text{pos}), \text{cut}(), \text{copy}(), \text{paste}(), \text{setSelection}(\text{start, end}), \text{moveCursor}(\text{pos})$
$\text{setAlignment}(), \text{setBulletList}(), \text{setIndentation}(), \text{insertImage}(), \text{insertTable}(), \text{insertLink}()$
$\text{replaceText}(), \text{replaceAll}(), \text{undo}(), \text{redo}() \ \ldots$

Insert and Delete Operations

Insert operation

$\text{ins}(p, c) = \text{insert character } c \text{ at position } p$

$\text{ins}(2, \text{X})$ applied to document abc yields new document abXc

Delete operation

$\text{del}(p) = \text{delete character at position } p$

$\text{del}(1)$ applied to document abc yields new document ac

For sake of simplicity, let's only consider Insert and Delete Operations

Example



Example

Alice intentions is to delete the A



Example

Alice intentions is to delete the A

Alice

ABCD

$\mathrm{del}(0)$

BCD

Bob

ABCD

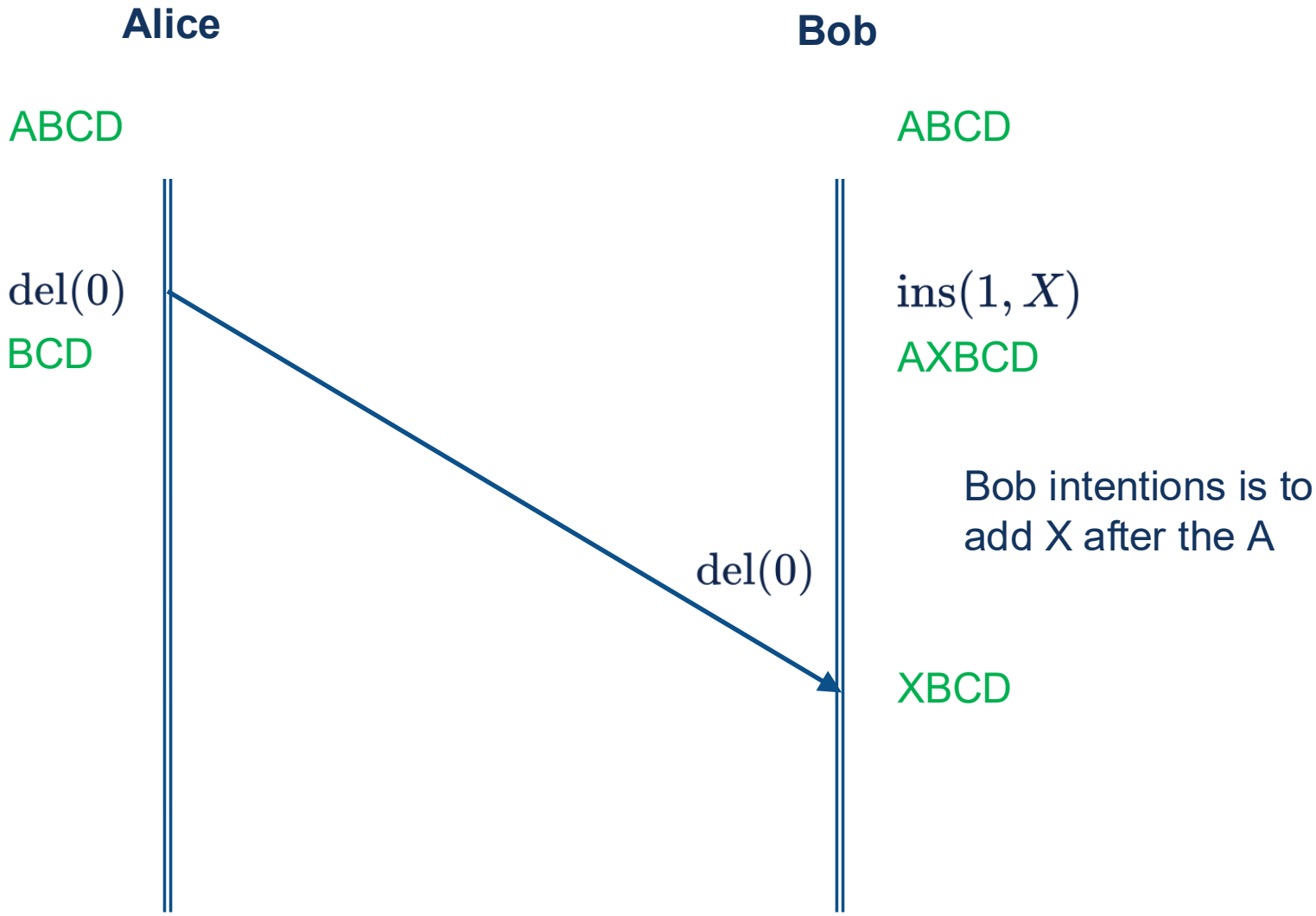
$\mathrm{ins}(1, X)$

AXBCD

Bob intentions is to add X after the A

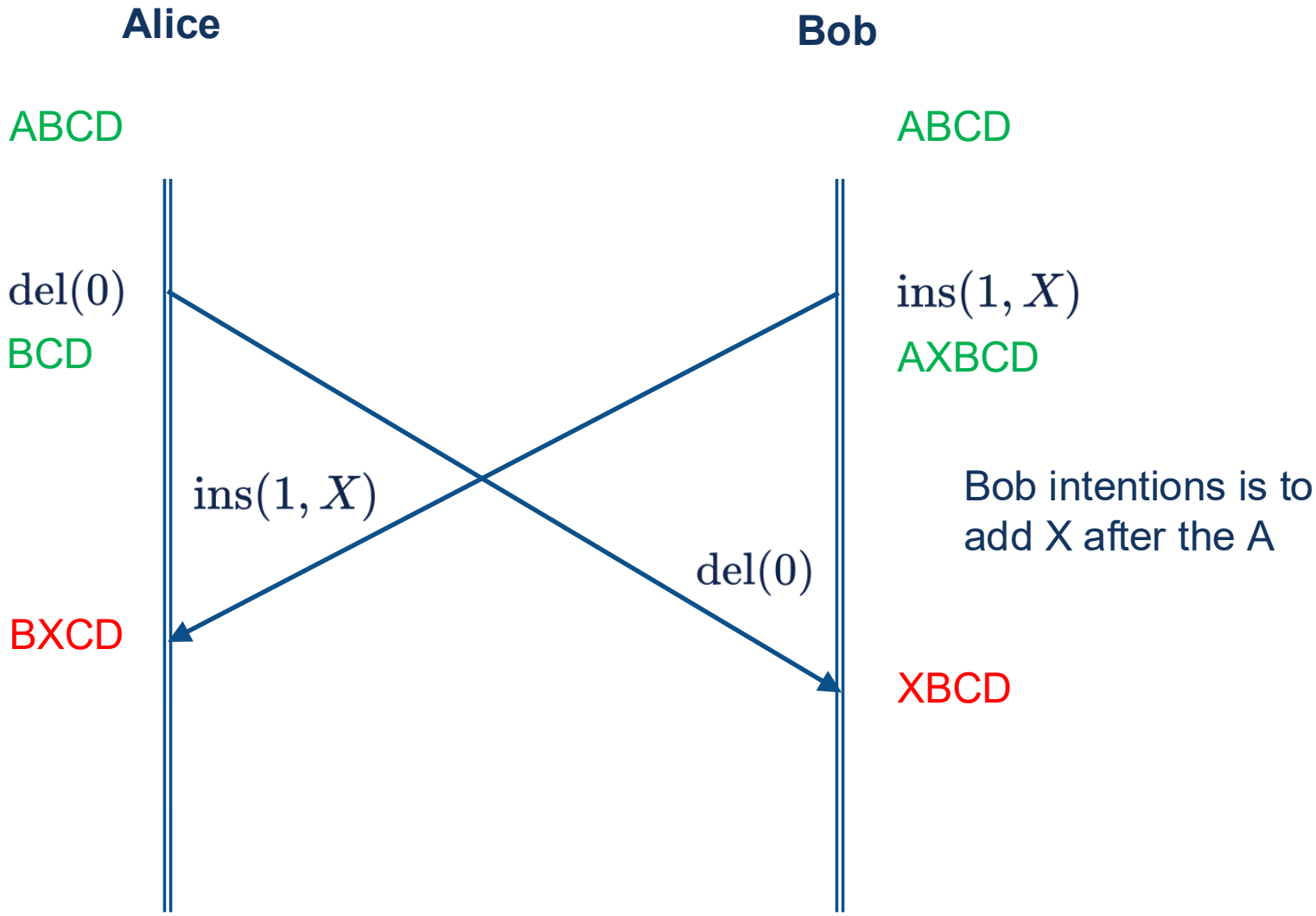
Example

Alice intentions is to delete the A



Example

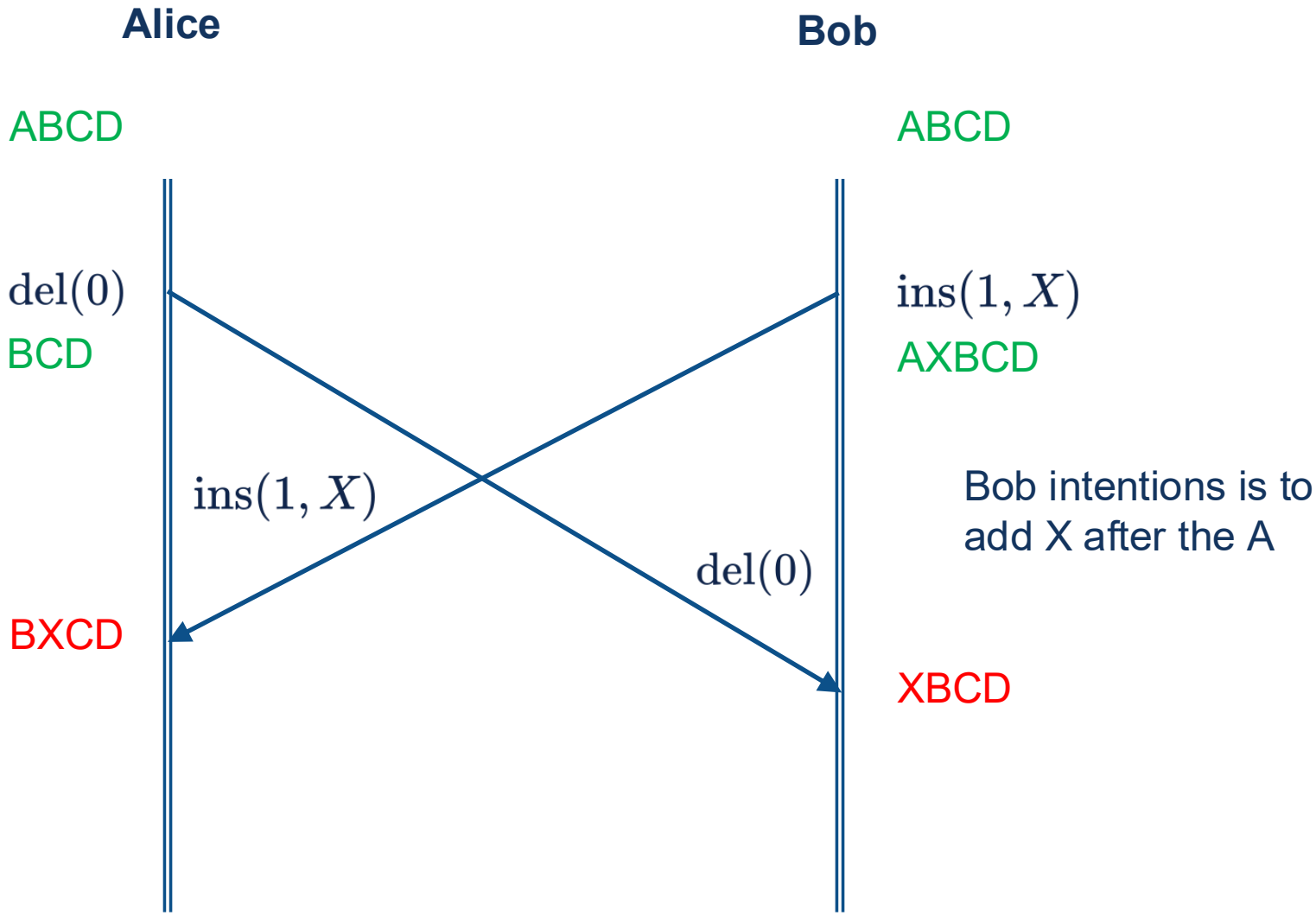
Alice intentions is to delete the A



Bob intentions is to add X after the A

Example

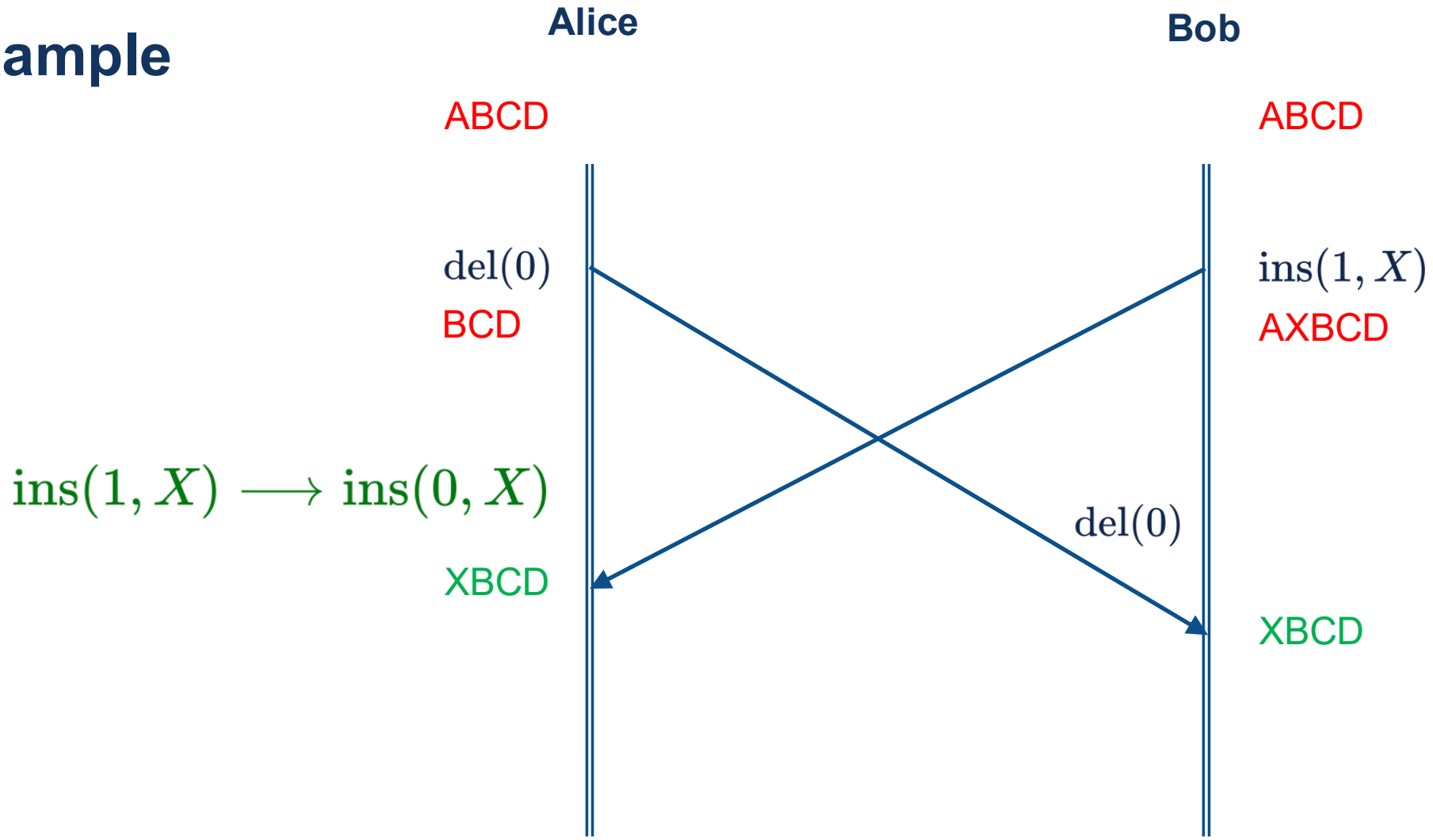
Alice intentions is to delete the A



Bob intentions is to add X after the A

CONFLICT: Documents has diverged, and Bob didn't want to insert an X after the B

Example



What about if we can transform the operation to enable the documents to converge , while also preserving user intentions?

Jupiter OT Algorithm

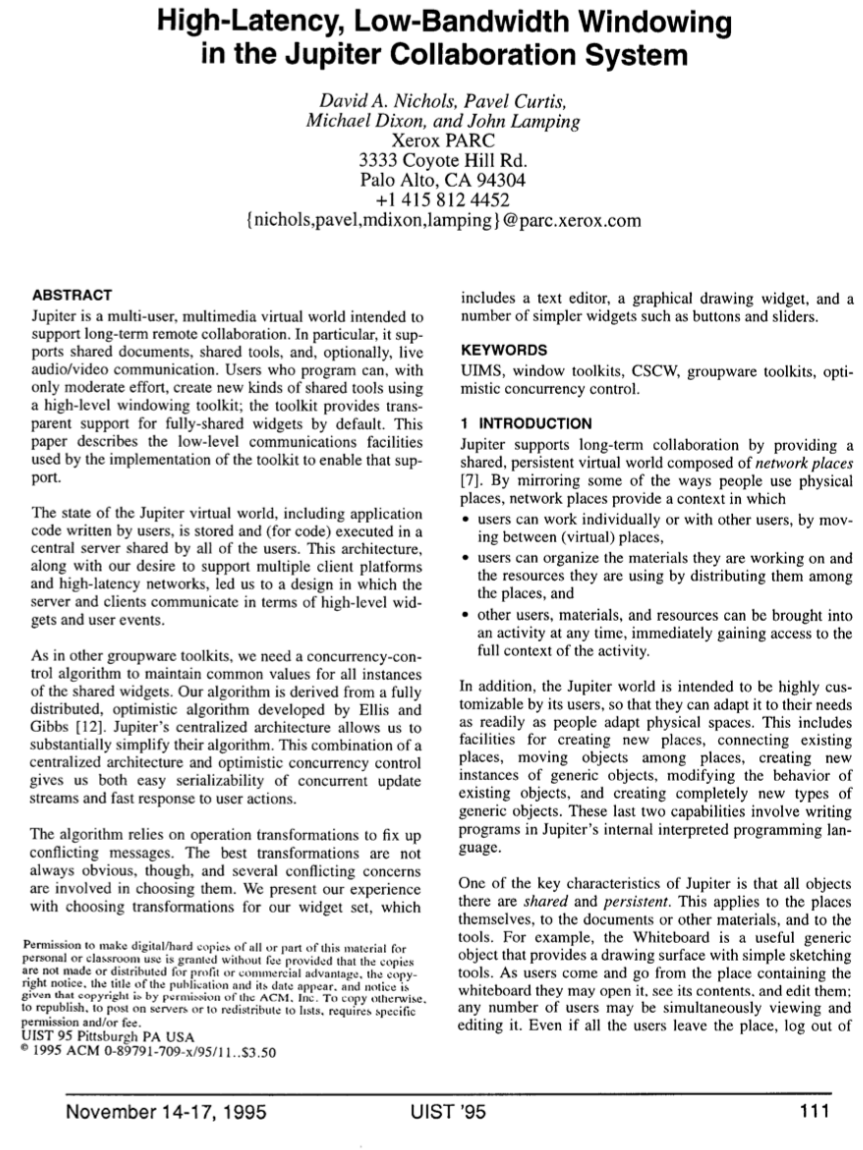
Operational Transformation algorithm
developed at Xerox PARC

Designed for **real-time collaborative editing** with **low latency and high scalability**

Two-Way Transformation

- **Client-side OT:** Transforms incoming remote operations against local unacknowledged operations
- **Server-side OT:** Transforms client operations against concurrently received ops from other clients.

Used in Google Docs (early version), Etherpad, Apache Wave



High-Latency, Low-Bandwidth Windowing in the Jupiter Collaboration System

David A. Nichols, Pavel Curtis, Michael Dixon, and John Lamping
Xerox PARC
3333 Coyote Hill Rd.
Palo Alto, CA 94304
+1 415 812 4452
{nichols,pavel,mdixon,lamping}@parc.xerox.com

ABSTRACT

Jupiter is a multi-user, multimedia virtual world intended to support long-term remote collaboration. In particular, it supports shared documents, shared tools, and, optionally, live audio/video communication. Users who program can, with only moderate effort, create new kinds of shared tools using a high-level windowing toolkit; the toolkit provides transparent support for fully-shared widgets by default. This paper describes the low-level communications facilities used by the implementation of the toolkit to enable that support.

The state of the Jupiter virtual world, including application code written by users, is stored and (for code) executed in a central server shared by all of the users. This architecture, along with our desire to support multiple client platforms and high-latency networks, led us to a design in which the server and clients communicate in terms of high-level widgets and user events.

As in other groupware toolkits, we need a concurrency-control algorithm to maintain common values for all instances of the shared widgets. Our algorithm is derived from a fully distributed, optimistic algorithm developed by Ellis and Gibbs [12]. Jupiter's centralized architecture allows us to substantially simplify their algorithm. This combination of a centralized architecture and optimistic concurrency control gives us both easy serializability of concurrent update streams and fast response to user actions.

The algorithm relies on operation transformations to fix up conflicting messages. The best transformations are not always obvious, though, and several conflicting concerns are involved in choosing them. We present our experience with choosing transformations for our widget set, which includes a text editor, a graphical drawing widget, and a number of simpler widgets such as buttons and sliders.

KEYWORDS

UIMS, window toolkits, CSCW, groupware toolkits, optimistic concurrency control.

1 INTRODUCTION

Jupiter supports long-term collaboration by providing a shared, persistent virtual world composed of *network places* [7]. By mirroring some of the ways people use physical places, network places provide a context in which

- users can work individually or with other users, by moving between (virtual) places,
- users can organize the materials they are working on and the resources they are using by distributing them among the places, and
- other users, materials, and resources can be brought into an activity at any time, immediately gaining access to the full context of the activity.

In addition, the Jupiter world is intended to be highly customizable by its users, so that they can adapt it to their needs as readily as people adapt physical spaces. This includes facilities for creating new places, connecting existing places, moving objects among places, creating new instances of generic objects, modifying the behavior of existing objects, and creating completely new types of generic objects. These last two capabilities involve writing programs in Jupiter's internal interpreted programming language.

One of the key characteristics of Jupiter is that all objects there are *shared* and *persistent*. This applies to the places themselves, to the documents or other materials, and to the tools. For example, the Whiteboard is a useful generic object that provides a drawing surface with simple sketching tools. As users come and go from the place containing the whiteboard they may open it, see its contents, and edit them; any number of users may be simultaneously viewing and editing it. Even if all the users leave the place, log out of

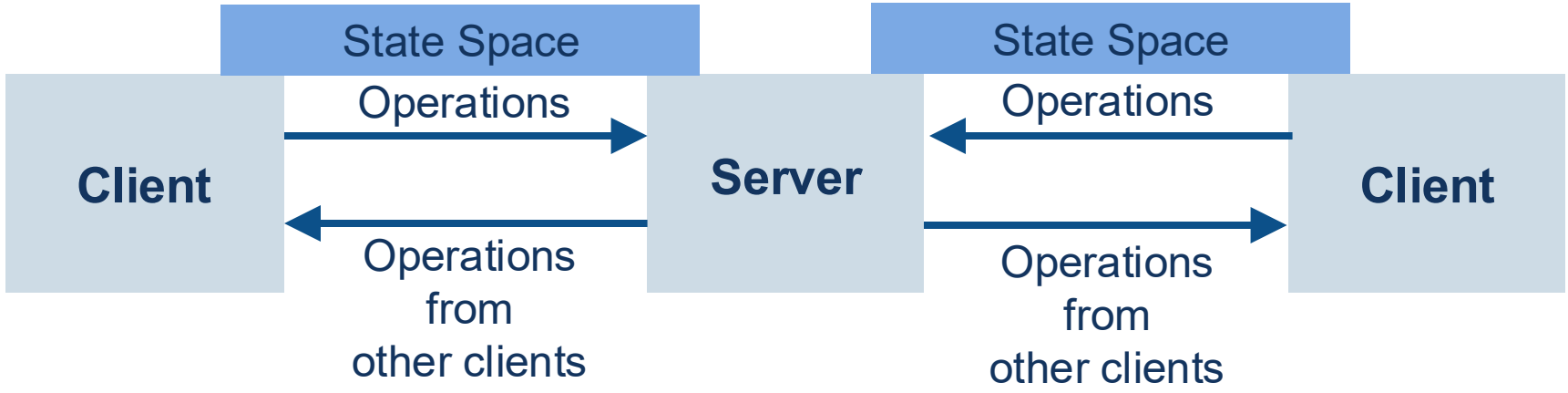
Permission to make digital/hard copies of all or part of this material for personal or classroom use is granted without fee provided that the copies are not made or distributed for profit or commercial advantage, the copyright notice, the title of the publication and its date appear, and notice is given that copyright is by permission of the ACM, Inc. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires specific permission and/or fee.
UIST 95 Pittsburgh PA USA
© 1995 ACM 0-89791-709-x/95/11..$3.50

Proceedings of the 8th Annual ACM Symposium on User Interface and Software Technology

Jupiter OT Algorithm

State Space refers to the set of possible states (possible configurations) that represent the document at various points during collaborative editing

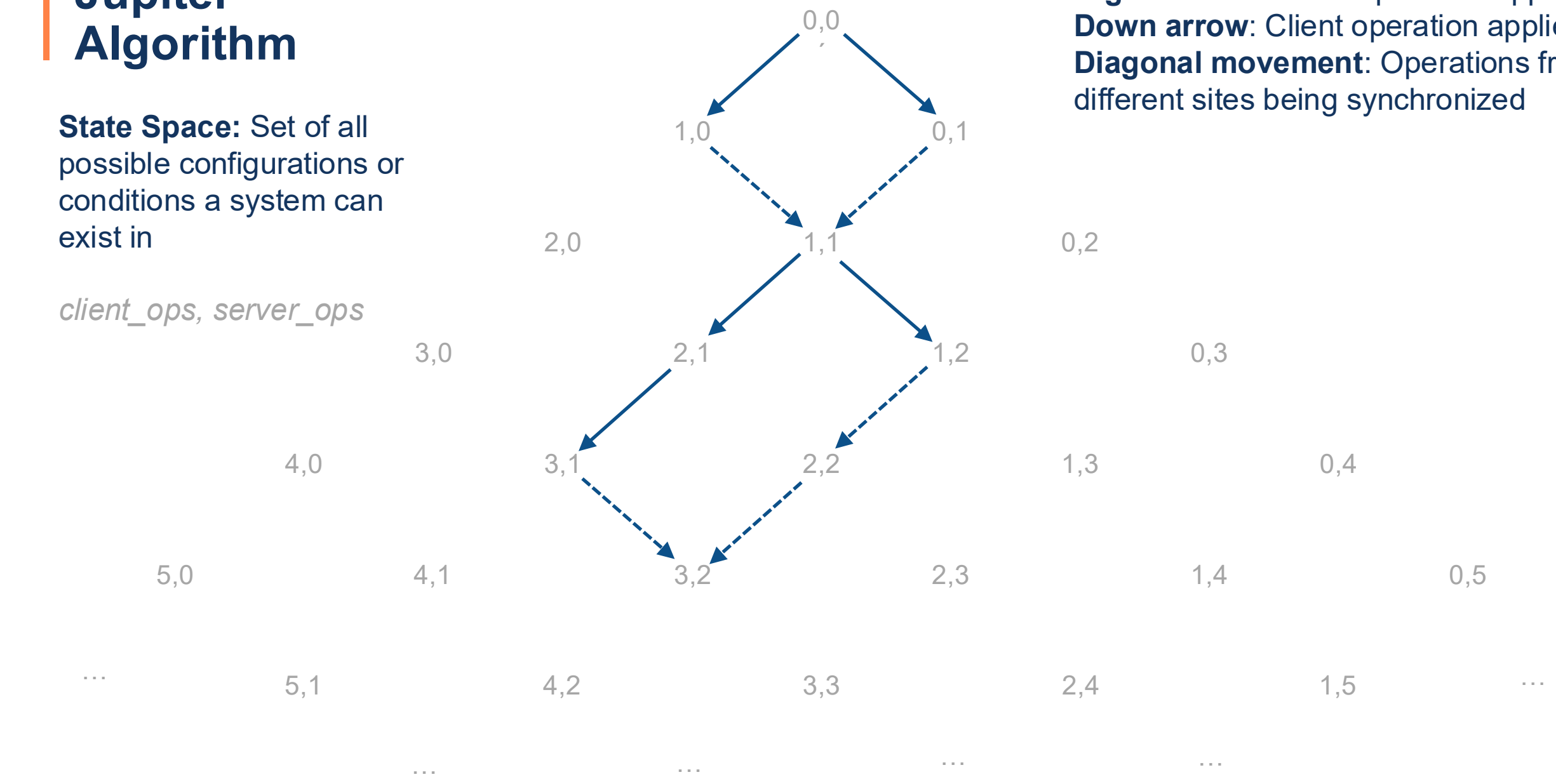
Each Client has its own local State Space and the Server has a corresponding State Space for each connected Client



Jupiter Algorithm

State Space: Set of all possible configurations or conditions a system can exist in

client_ops, server_ops



Right arrow: Server operation applied
Down arrow: Client operation applied
Diagonal movement: Operations from different sites being synchronized

Ins and Del Tranformation Functions

The Transformation Function $T : \mathrm{O}_{\text{target}} \times \mathrm{O}_{\text{ref}} \rightarrow \mathrm{O}^{T}_{\text{target}}$ adjusts the position of operation $\mathrm{O}_{\text{target}}$ to account for the positional changes caused by operation O_{ref}, ensuring it can be applied correctly while preserving its original intentions

$$T(\mathrm{ins}(p_1, c_1), \mathrm{ins}(p_2, c_2)) = \begin{cases} \mathrm{ins}(p_1, c_1) & \text{if } p_1 \leq p_2 \\ \mathrm{ins}(p_1 + 1, c_1) & \text{if } p_1 > p_2 \end{cases}$$

$$T(\mathrm{ins}(p_1, c), \mathrm{del}(p_2)) = \begin{cases} \mathrm{ins}(p_1, c) & \text{if } p_1 \leq p_2 \\ \mathrm{ins}(p_1 - 1, c) & \text{if } p_1 > p_2 \end{cases}$$

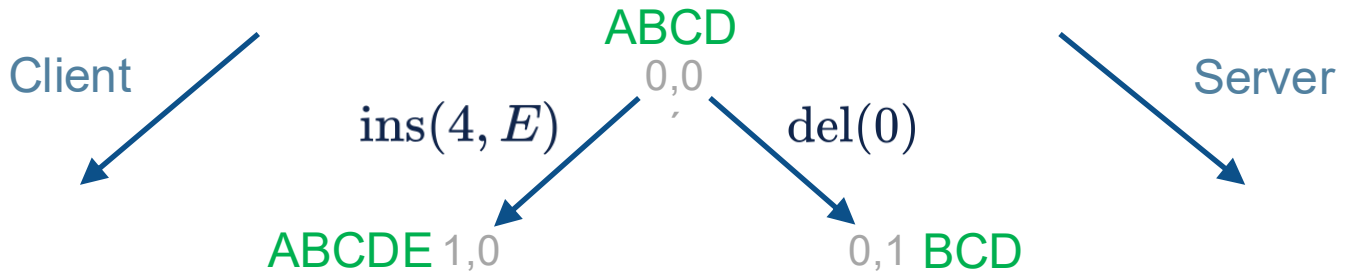
$$T(\mathrm{del}(p_1), \mathrm{ins}(p_2, c)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 + 1) & \text{if } p_1 \geq p_2 \end{cases}$$

$$T(\mathrm{del}(p_1), \mathrm{del}(p_2)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 - 1) & \text{if } p_1 > p_2 \\ \mathrm{nop}() & \text{if } p_1 = p_2 \end{cases}$$

Jupiter Algorithm

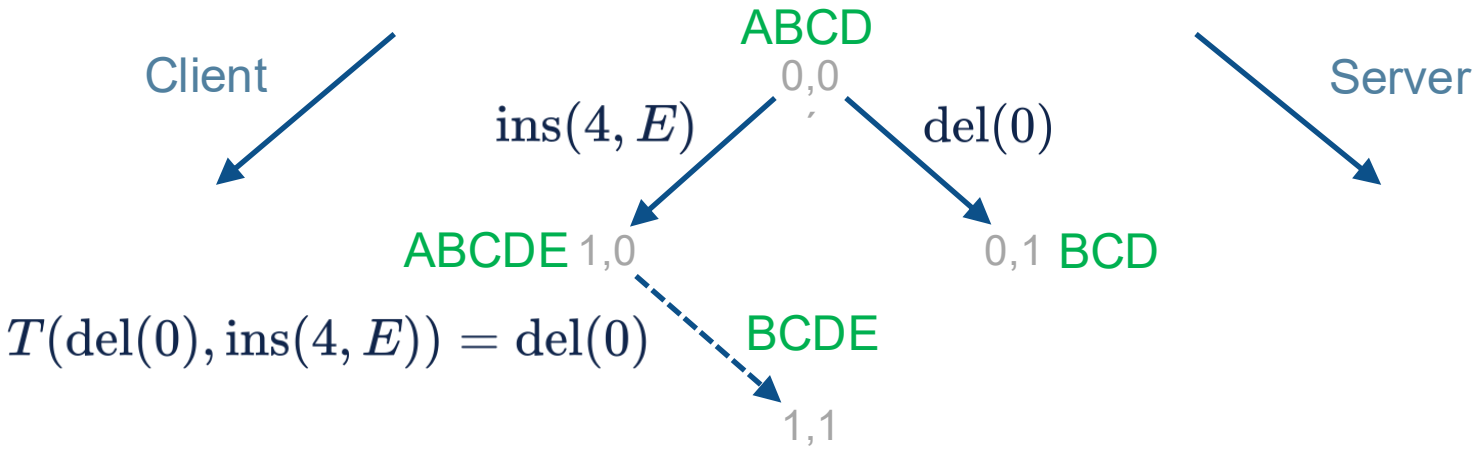


Jupiter Algorithm



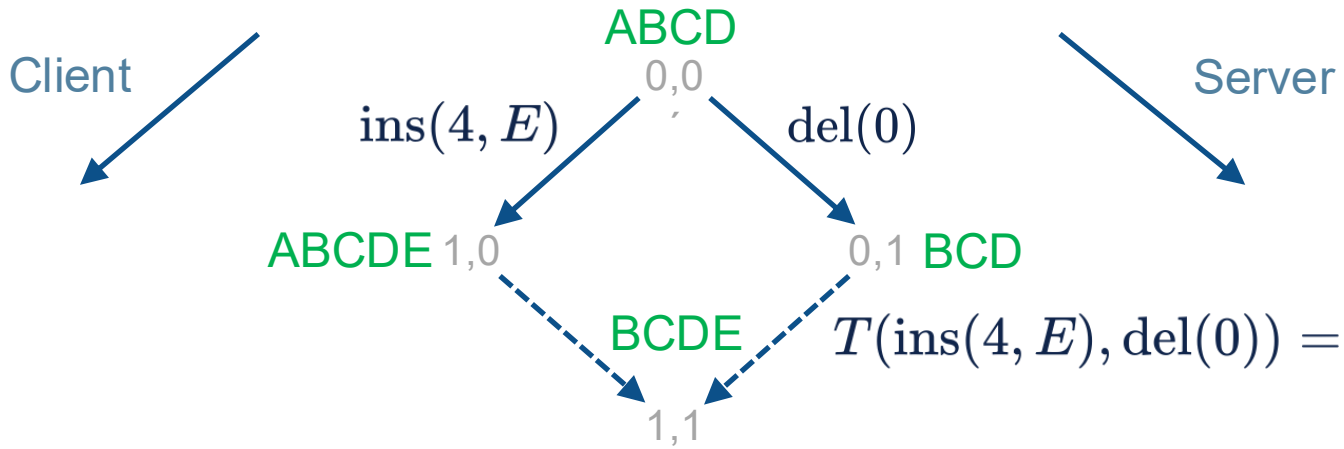
Client and Server (via other clients) edits the text simultaneously

Jupiter Algorithm



$$T(\mathrm{del}(p_1), \mathrm{ins}(p_2, c)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 + 1) & \text{if } p_1 \geq p_2 \end{cases}$$

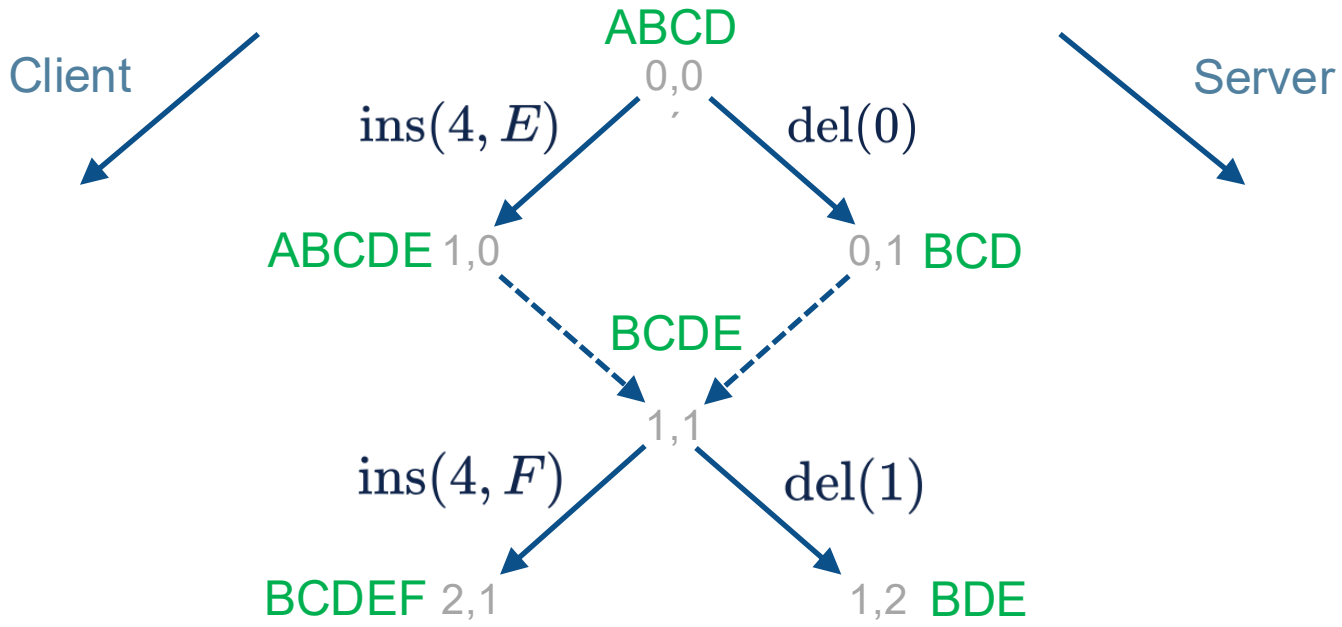
Jupiter Algorithm



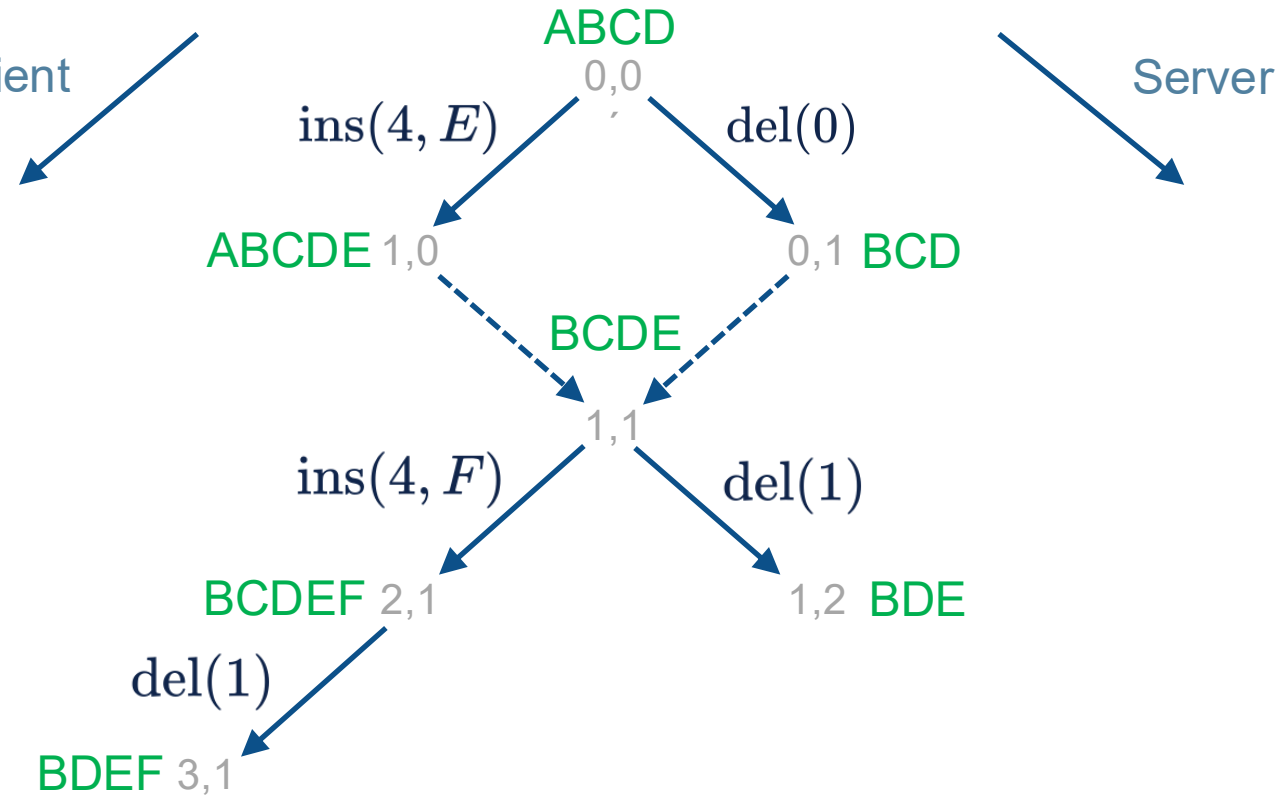
$$T(\mathrm{ins}(4, E), \mathrm{del}(0)) = \mathrm{ins}(3, E)$$

$$T(\mathrm{ins}(p_1, c), \mathrm{del}(p_2)) = \begin{cases} \mathrm{ins}(p_1, c) & \text{if } p_1 \leq p_2 \\ \mathrm{ins}(p_1 - 1, c) & \text{if } p_1 > p_2 \end{cases}$$

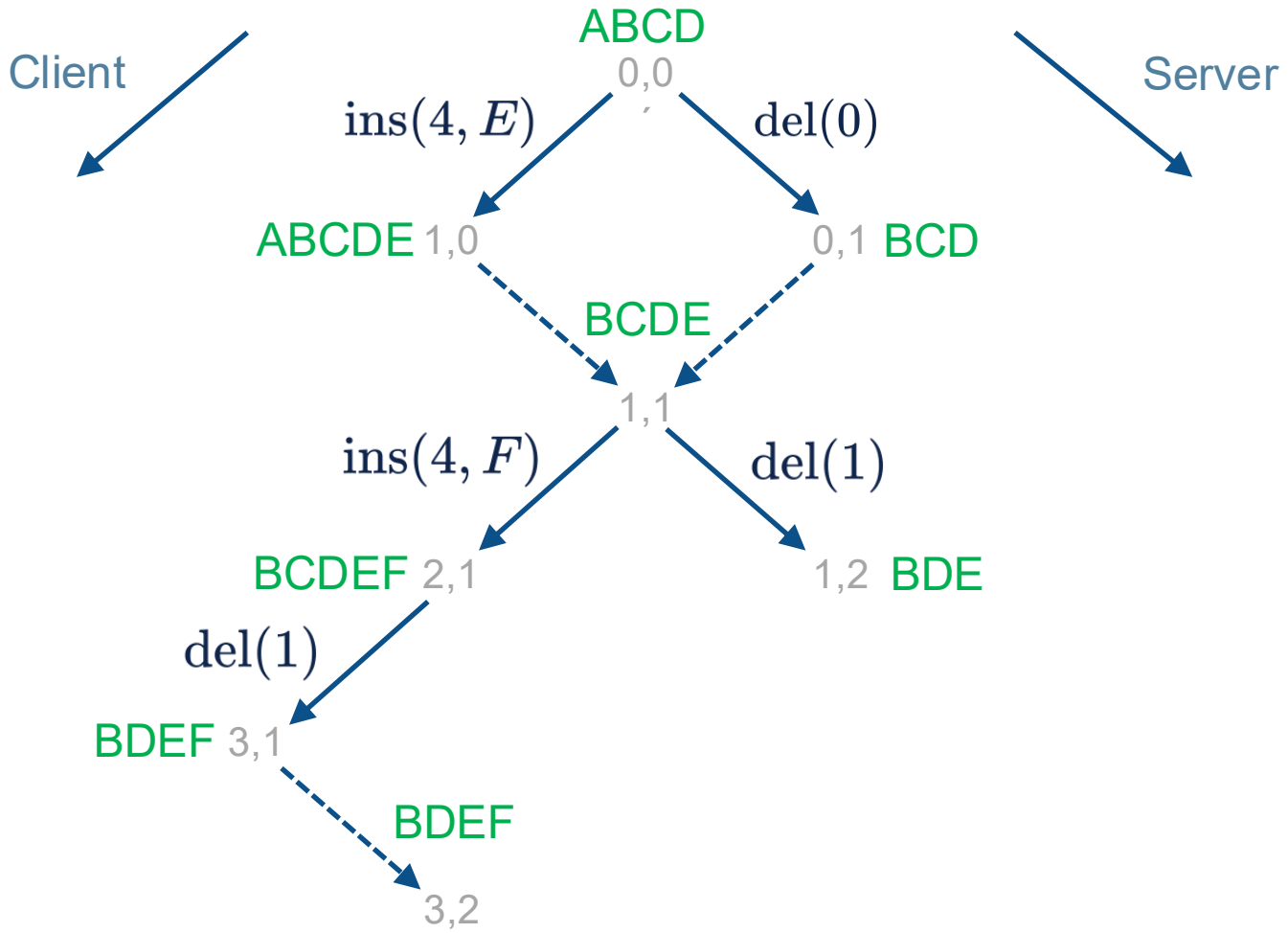
Jupiter Algorithm



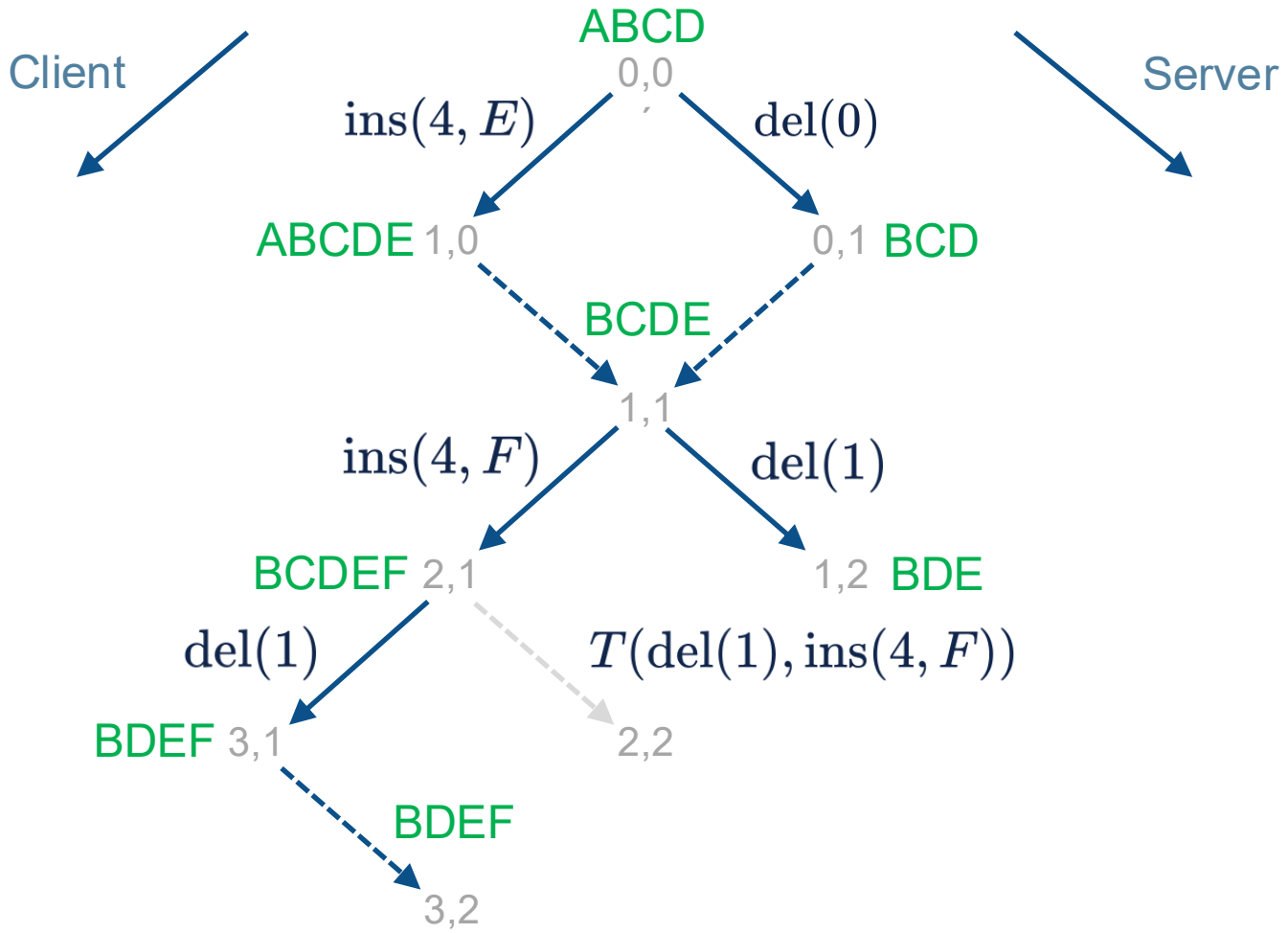
Jupiter Algorithm



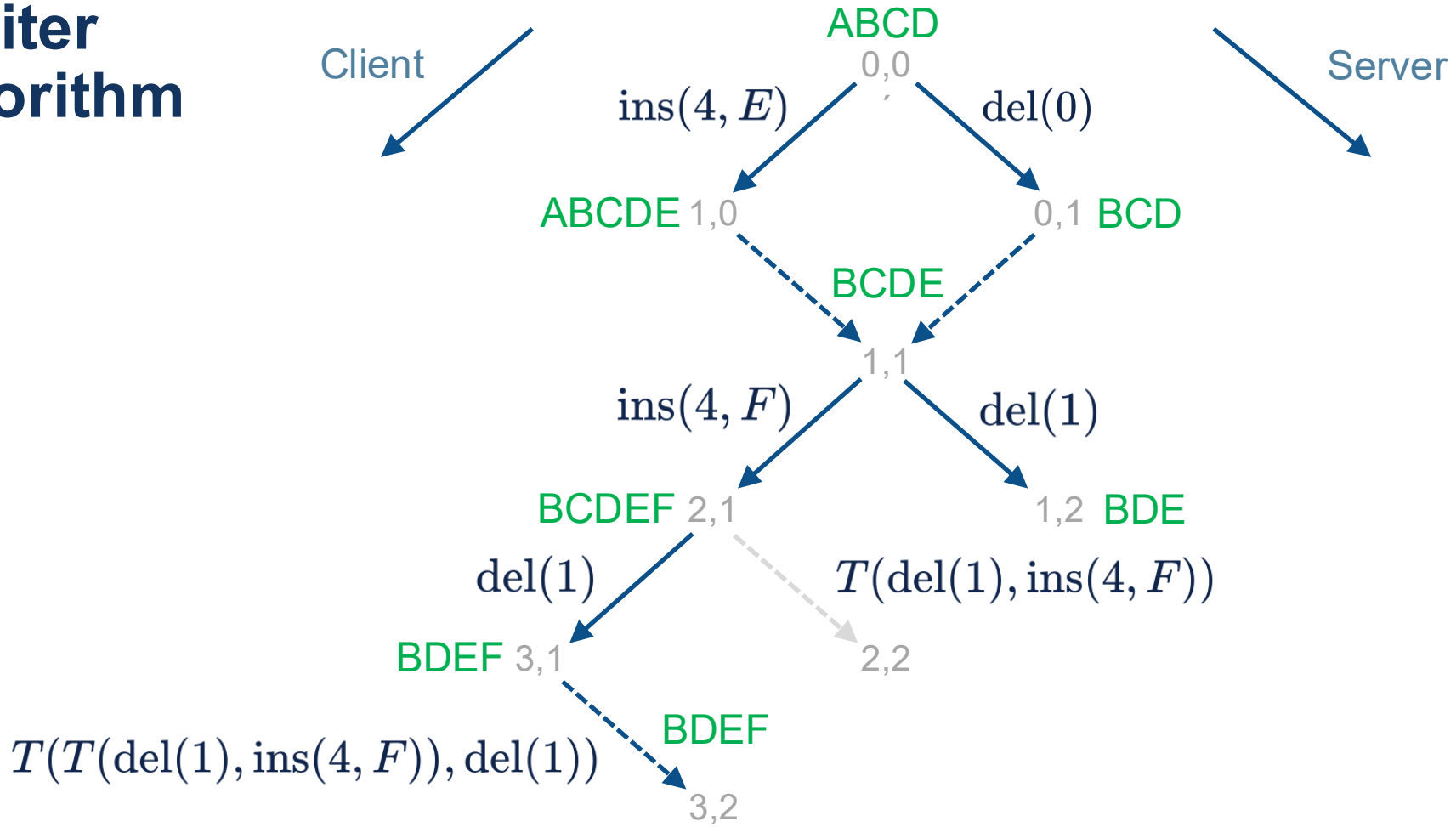
Jupiter Algorithm



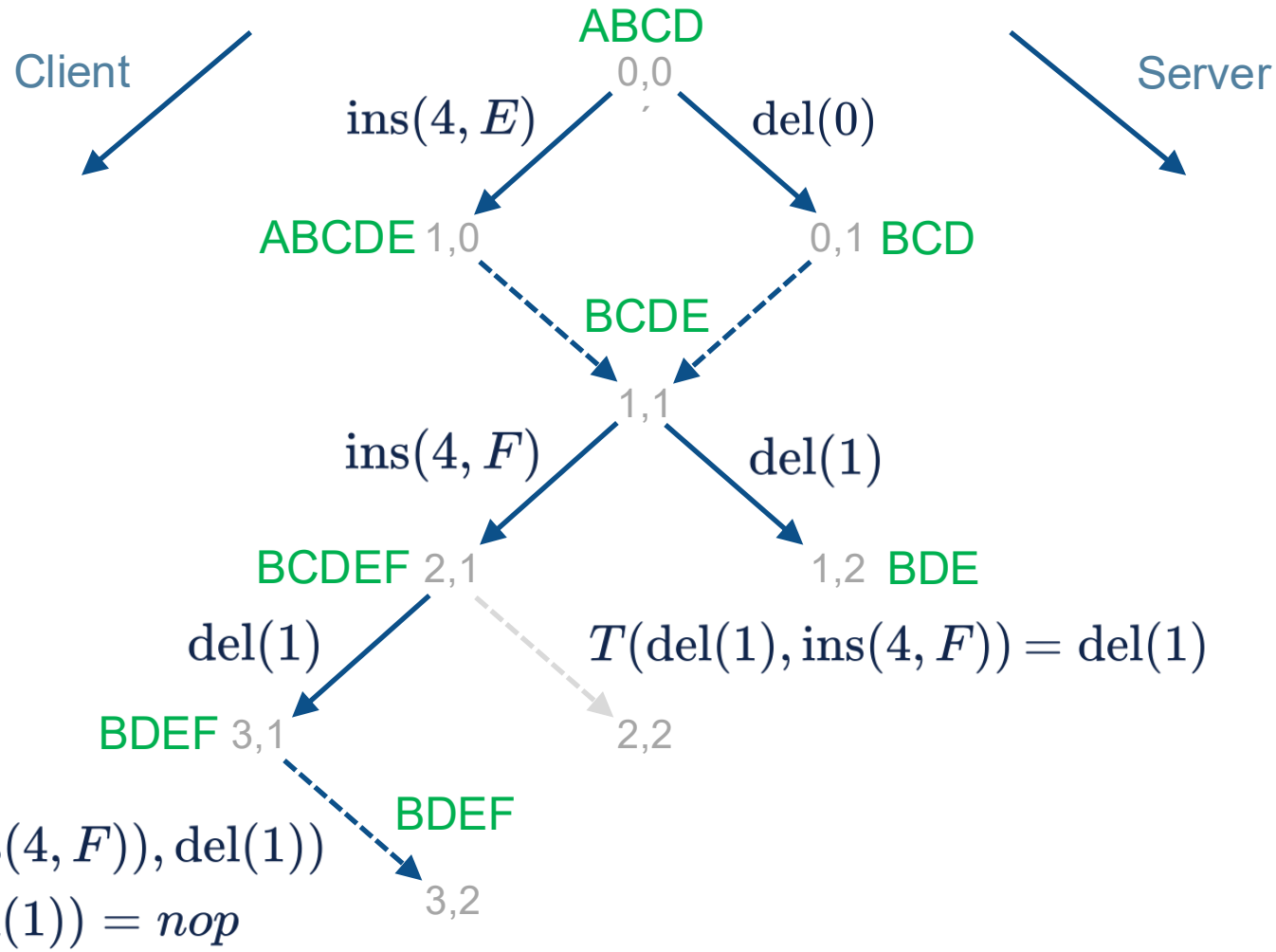
Jupiter Algorithm



Jupiter Algorithm



Jupiter Algorithm

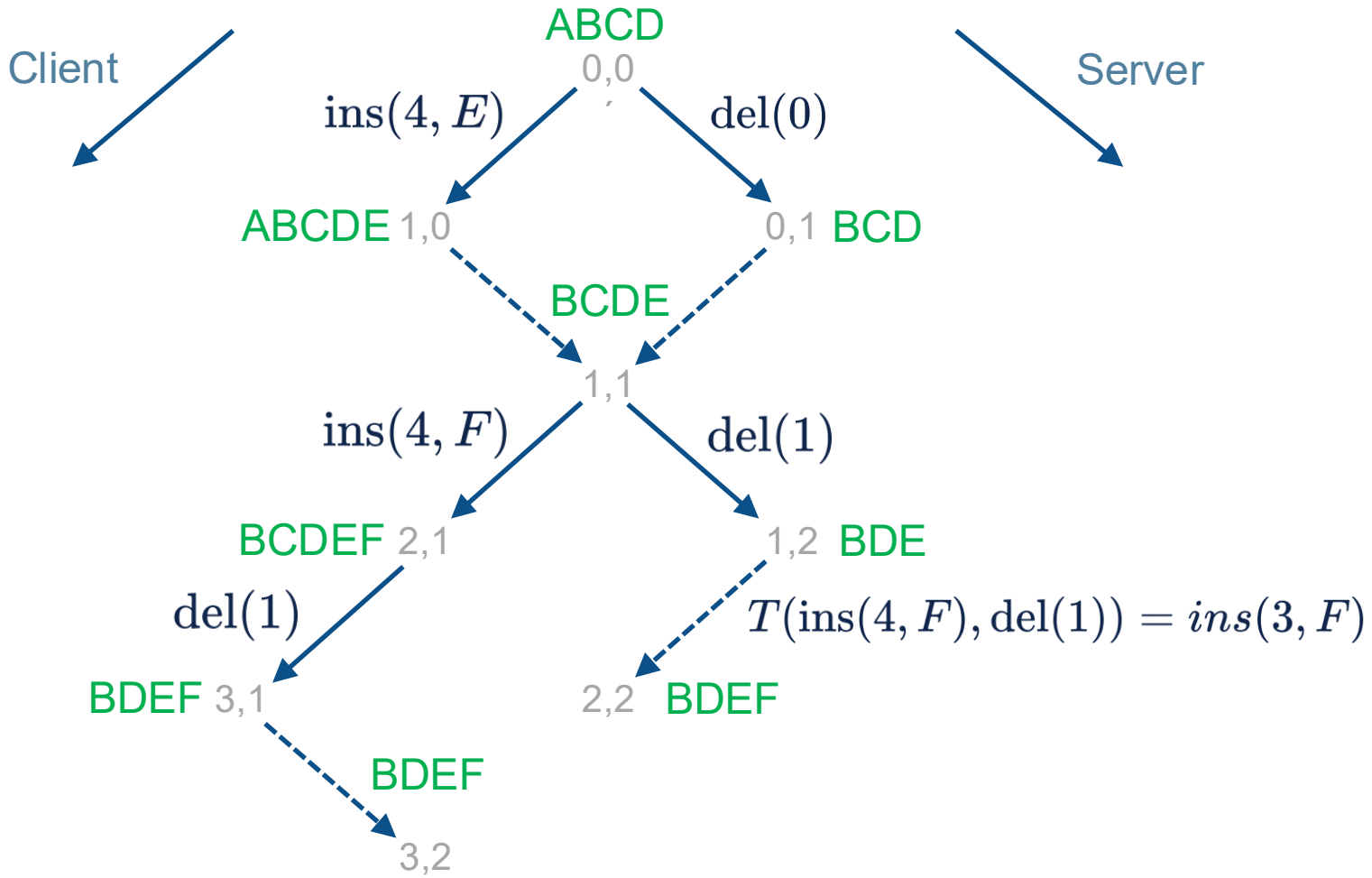


$$T(T(\mathrm{del}(1), \mathrm{ins}(4, F)), \mathrm{del}(1)) = T(\mathrm{del}(1), \mathrm{del}(1)) = nop$$

$$T(\mathrm{del}(p_1), \mathrm{del}(p_2)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 - 1) & \text{if } p_1 > p_2 \\ \mathrm{nop}() & \text{if } p_1 = p_2 \end{cases}$$

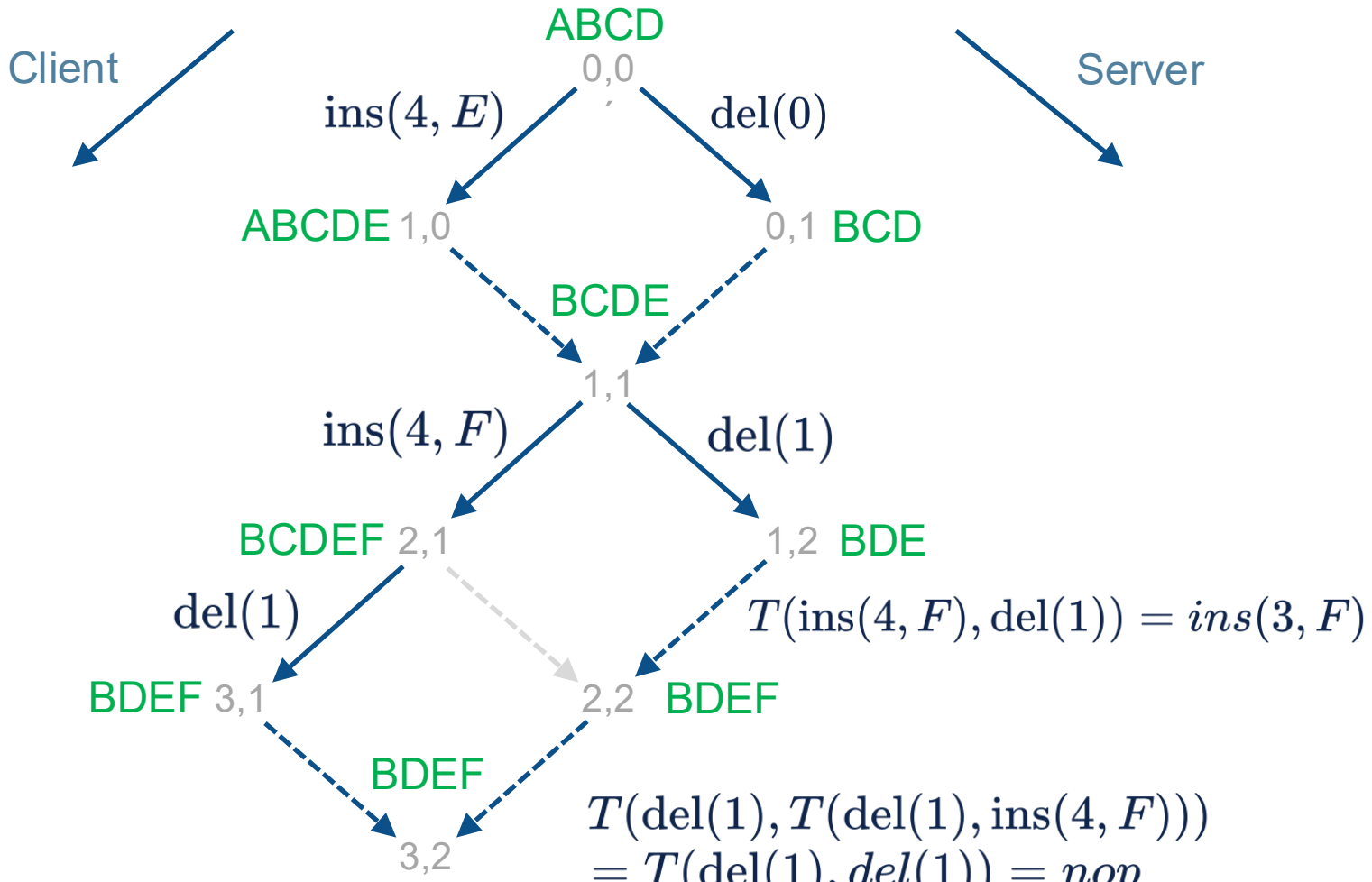
$$T(\mathrm{del}(p_1), \mathrm{ins}(p_2, c)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 + 1) & \text{if } p_1 \geq p_2 \end{cases}$$

Jupiter Algorithm



$$T(\mathrm{ins}(p_1, c), \mathrm{del}(p_2)) = \begin{cases} \mathrm{ins}(p_1, c) & \text{if } p_1 \leq p_2 \\ \mathrm{ins}(p_1 - 1, c) & \text{if } p_1 > p_2 \end{cases}$$

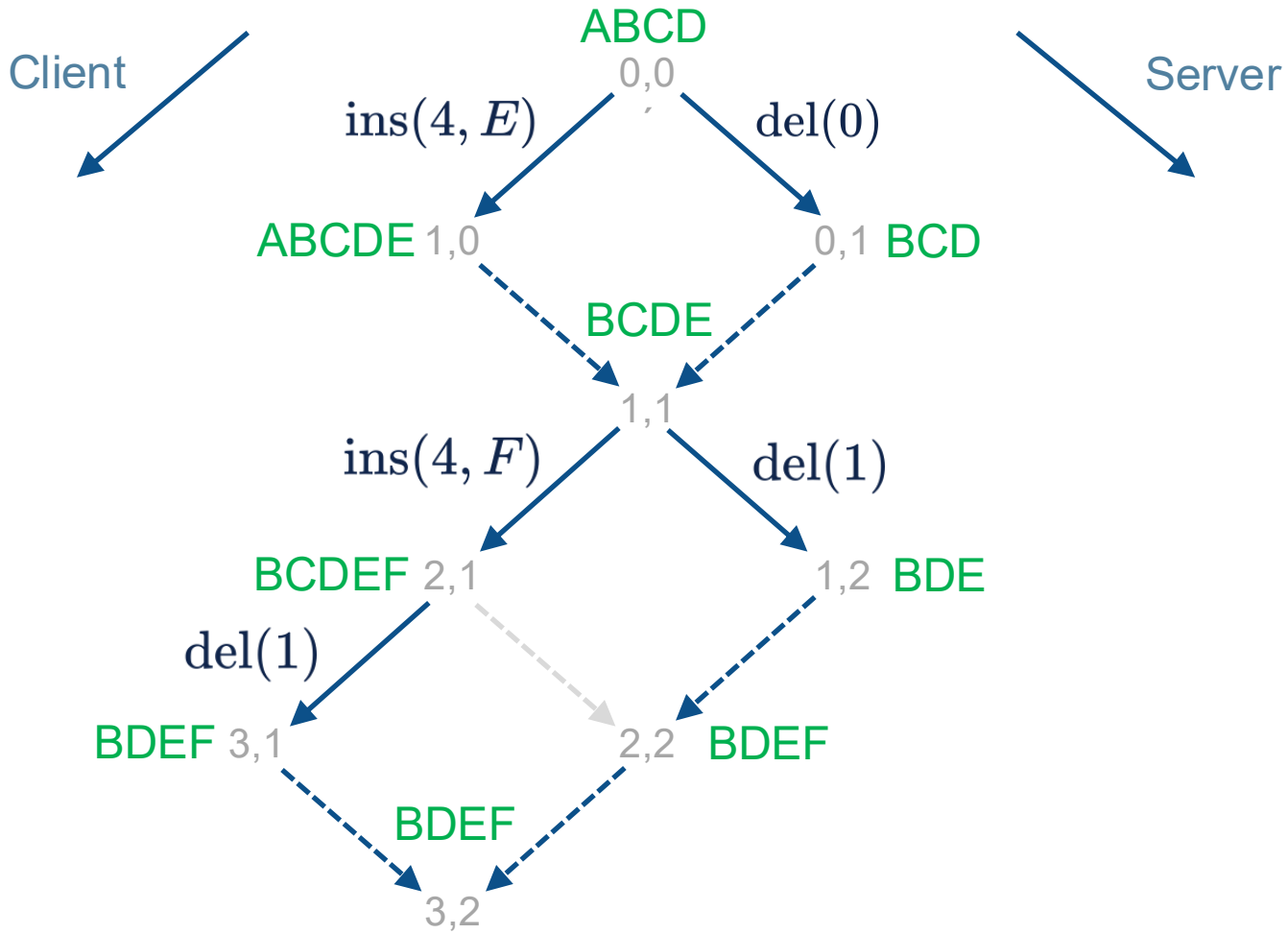
Jupiter Algorithm



$$T(\mathrm{del}(p_1), \mathrm{ins}(p_2, c)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 + 1) & \text{if } p_1 \geq p_2 \end{cases}$$

$$T(\mathrm{del}(p_1), \mathrm{del}(p_2)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 - 1) & \text{if } p_1 > p_2 \\ \mathrm{nop}() & \text{if } p_1 = p_2 \end{cases}$$

Jupiter Algorithm



Both Client and Server are synced at State 3,2

Can we use CRDT instead of OT?

Conflict-Free Replicated Datatypes (CRDT)

In traditional distributed systems, ***we need coordination (locks, consensus protocols)*** to avoid conflicts to handle concurrent changes

- This creates bottlenecks and failure points.

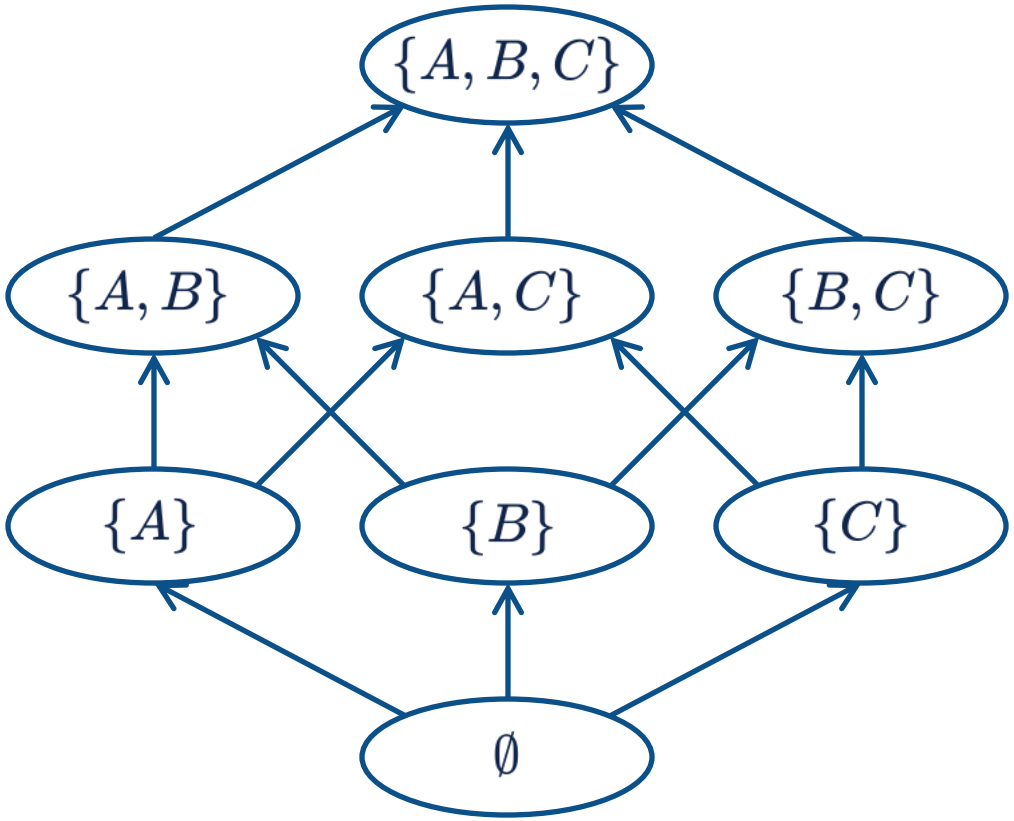
The CRDT Solution: Mathematical properties allow us to design data structures where conflicts are mathematically impossible

- All replicas will converge to the same state
- No information is lost during merging
- No coordination protocols are needed
- Network delays and failures can't break consistency

- **Use Cases:** Offline applications, multi-device sync, sync play lists …

Join-Semilattice

Partially ordered set that has a join *(a Least Upper Bound)* for any nonempty finite subset

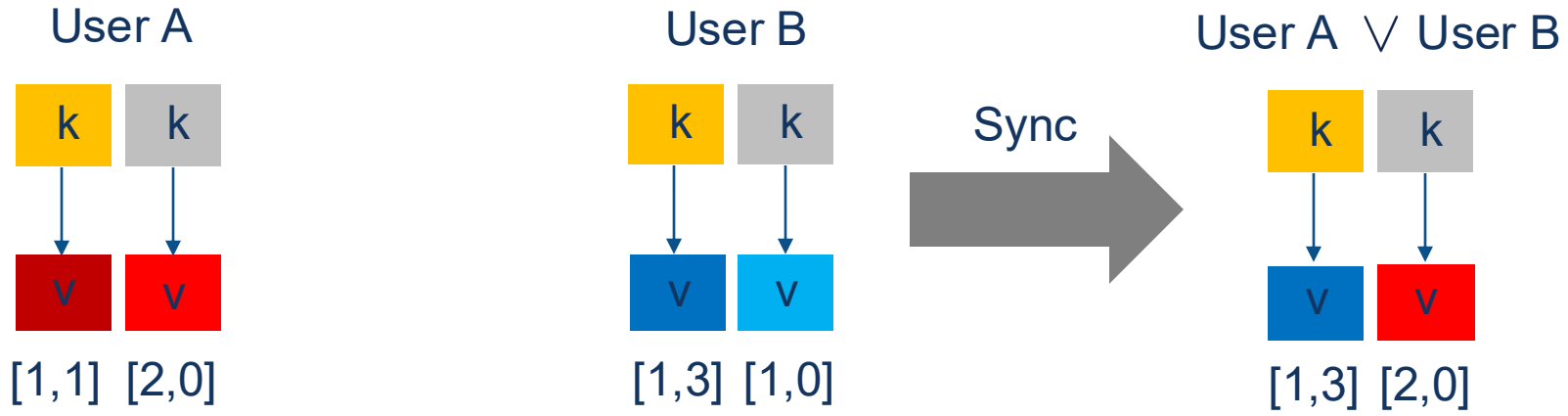


Conflict-free Merging

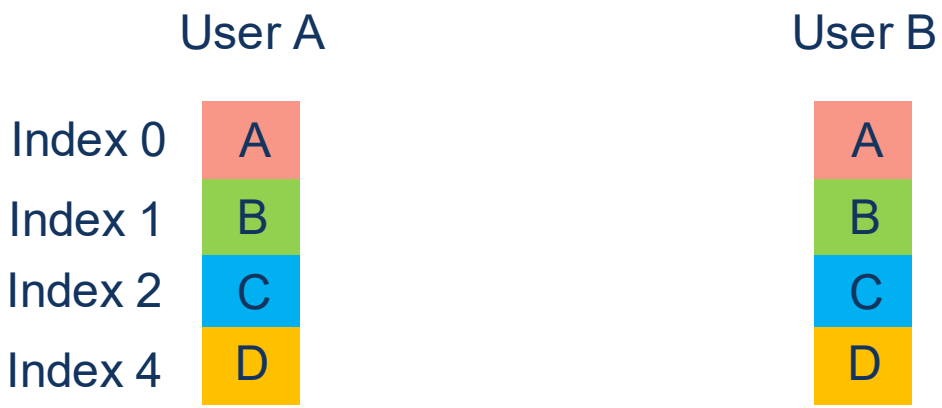
- Guaranteed convergence
- Order independence
- Fault tolerance
- No coordination needed

Associativity	$A \vee (B \vee C) = (A \vee B) \vee C$	Grouping doesn't matter
Commutativity	$A \vee B = B \vee A$	Order doesn't matter
Idempotence	$A \vee A = A$	Duplicate merges are safe

LWW Register



Sequential CRDT



Sequential CRDT

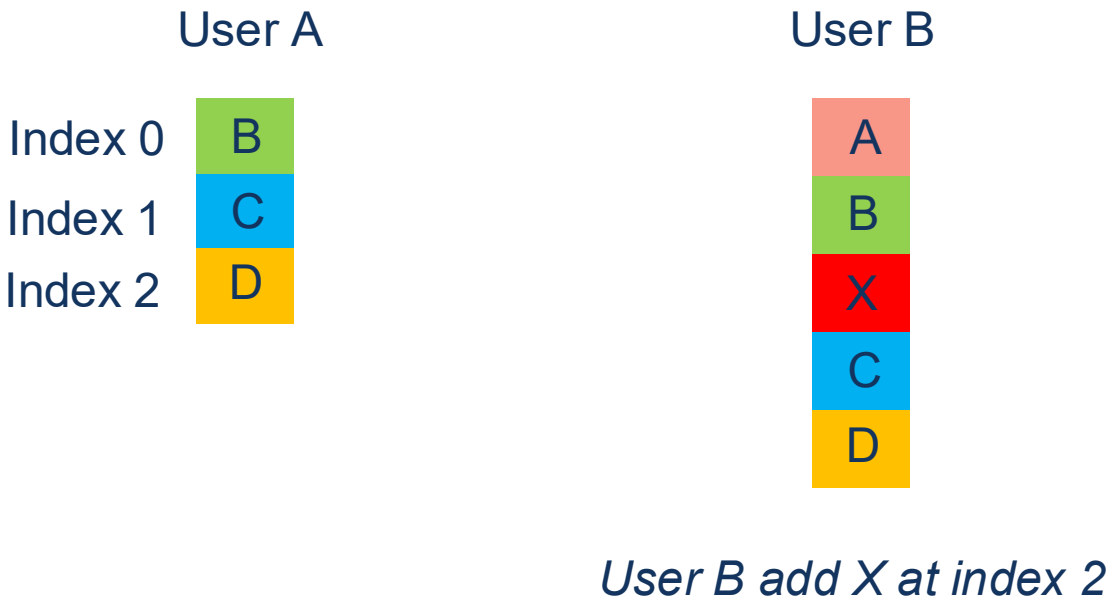


User A removes A at index 0

Sequential CRDT

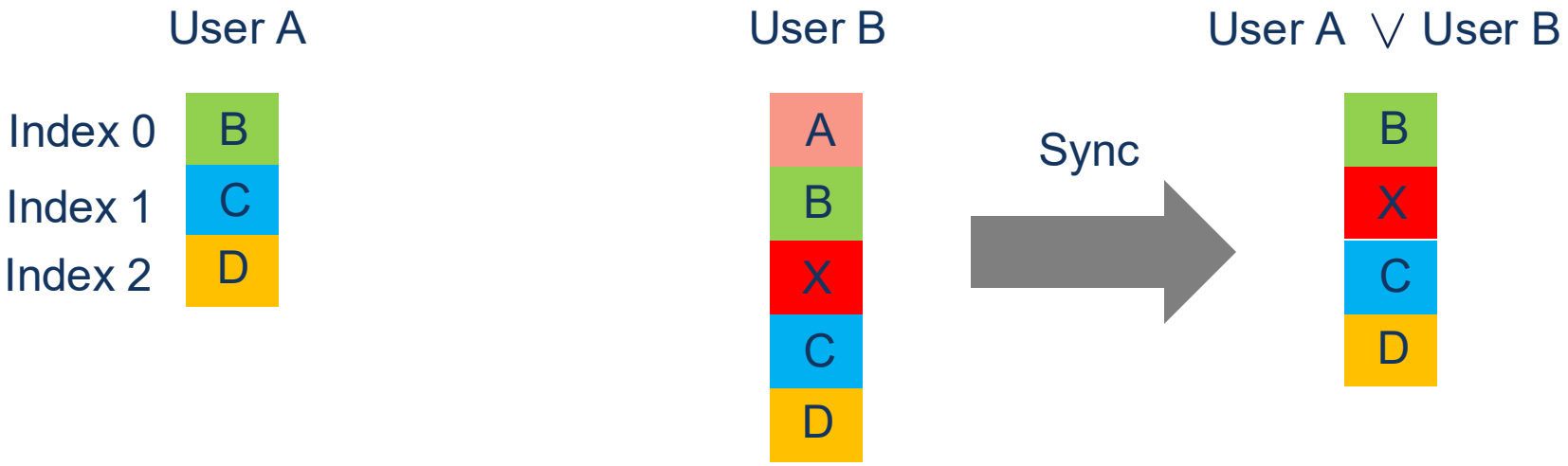


Sequential CRDT



User B add X at index 2

Sequential CRDT



Instead of transforming/changing the index can we use another type of index?

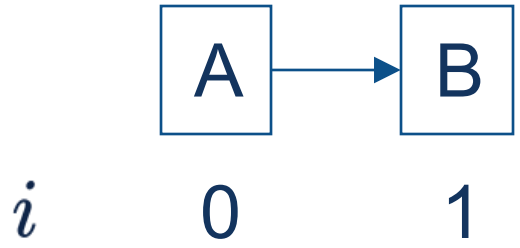
Problem with indices

$$i \in \{0, 1, 2, 3, \ldots, n\}$$



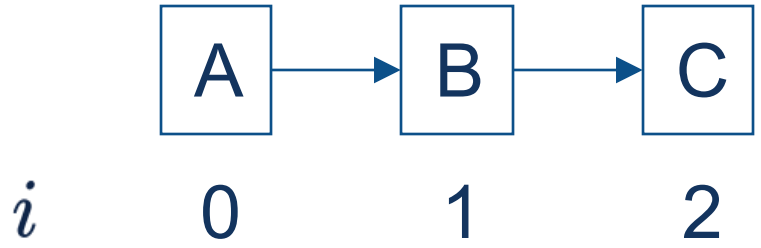
Problem with indices

$$i \in \{0, 1, 2, 3, \ldots, n\}$$



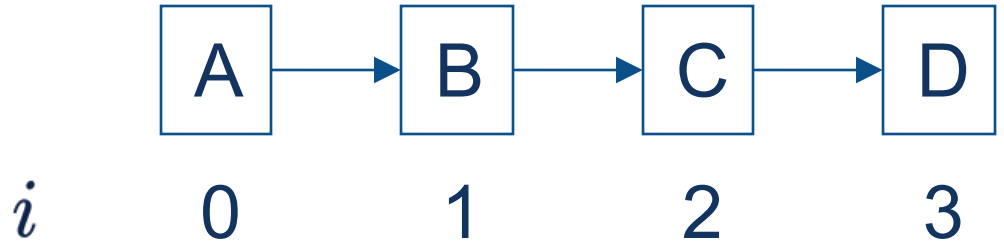
Problem with indices

$$i \in \{0, 1, 2, 3, \ldots, n\}$$



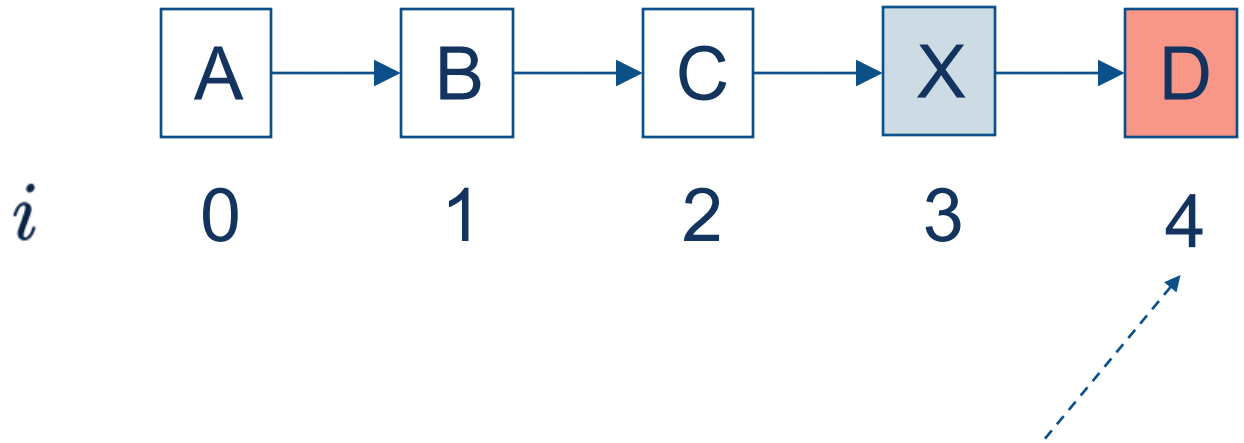
Problem with indices

$$i \in \{0, 1, 2, 3, \ldots, n\}$$

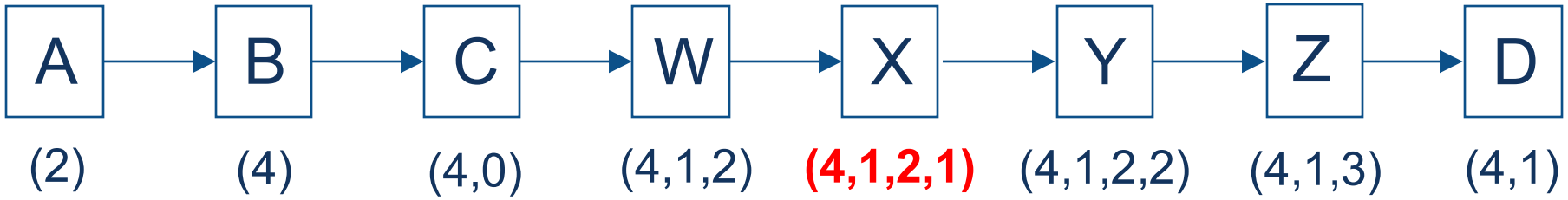


Problem with indices

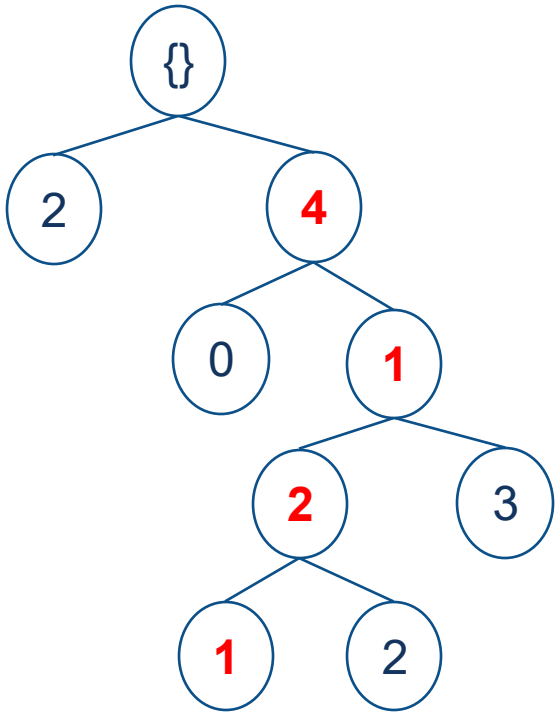
$$i \in \{0, 1, 2, 3, \ldots, n\}$$



Logoot/LSEQ (List Sequence CRDT)



$$\text{id} = (p_1, p_2, \ldots, p_k) \text{ is a tree path where } p_i \in \{0, 1, \ldots, \text{base}_i - 1\}$$



Unlike traditional array-based structures, **LSEQ** assign each element a **unique, immutable identifier**, which serves both as a **position** and as a **replica-independent order key**

Each new element is placed **between two existing IDs**
If there's no room at the current level, just **create a new subtree**
No **need to transform or shift other elements' IDs**.

Assume Base=5

LSEQ (List Sequence CRDT)

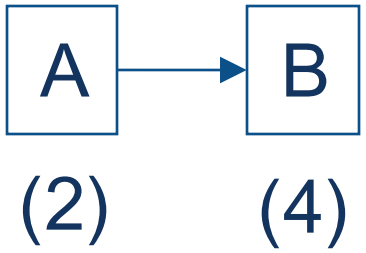
A

(2)

Randomly pick an ID between 0 and Base-1

Assume Base=5

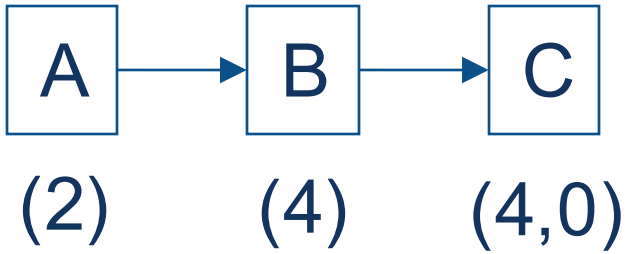
LSEQ (List Sequence CRDT)



Randomly pick an ID between 3 and Base-1

Assume Base=5

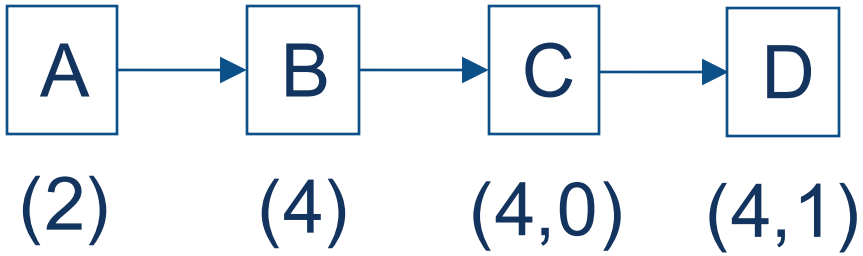
LSEQ (List Sequence CRDT)



We run out range, we need to split, randomly pick an id between 0 and Base-1

Assume Base=5

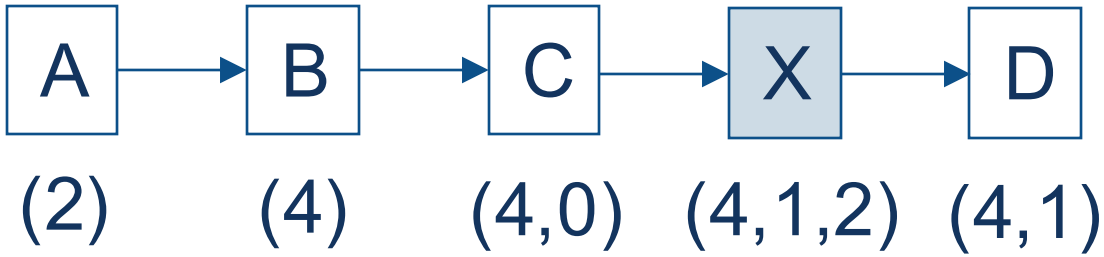
LSEQ (List Sequence CRDT)



Randomly pick an ID between 1 and Base-1

Assume Base=5

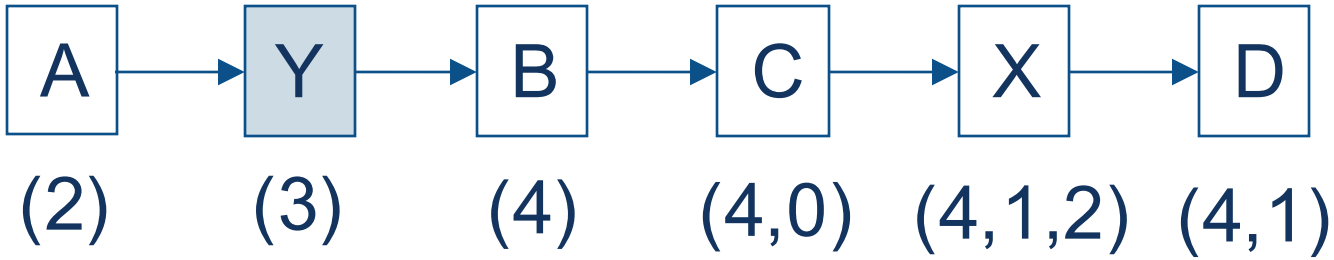
LSEQ (List Sequence CRDT)



We run out range, we need to split, randomly pick an id between 0 and Base-1

Assume Base=5

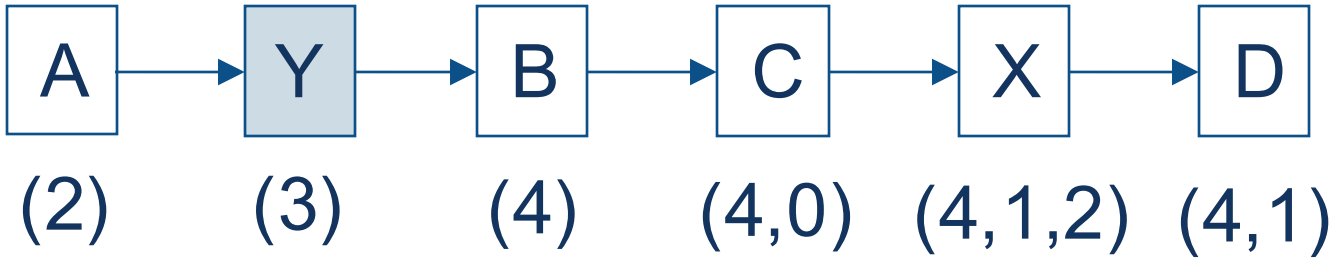
LSEQ (List Sequence CRDT)



Randomly pick an id between 3 and Base-1
Only 3 is possible

Assume Base=5

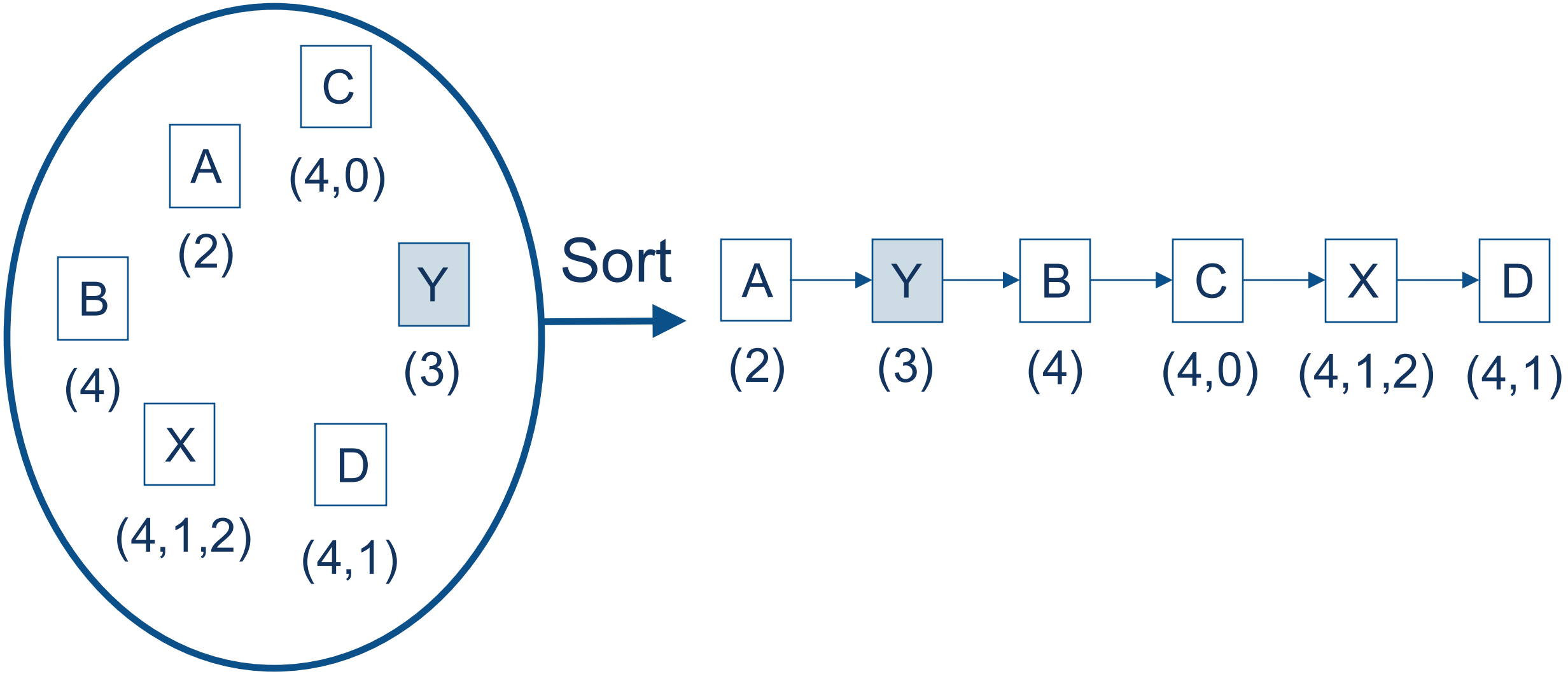
LSEQ (List Sequence CRDT)



Randomly pick an id between 3 and Base-1
Only 3 is possible

Assume Base=5

LSEQ (List Sequence CRDT)



Splitting Rules

Left	Right	Result (Example)	Notes
[1]	[4]	[2]	Simple gap at level 0
[2]	[4]	[3]	One or more values between
[3]	[4]	[3, 0]	No gap → go to level 1
[]	[2]	[0] or [1]	Insert at beginning
[4]	[]	[5, 0]	No room at level 0 → go deeper
[2]	[2, 3]	[2, 1], [2, 2]	Inserting before deeper right
[2, 0]	[2, 4]	[2, 1] to [2, 3]	Range exists at level 1
[2, 4]	[3]	[2, 4, 0]	No gap → go to level 2
[0]	[4]	[1], [2], [3]	Several options at level 0
[3, 4]	[3, 4]	[3, 4, 0]	Identical IDs → extend to next level

CRDT Syncing

User A

ABCD

User B

ABCD

CRDT Syncing

User A

ABCD

Insert E after D

$$\{(A,(2)),\ (B,(4)),\ (C,(4,0)),\ (D,(4,1))\} \vee \{(E,(4,2))\}$$

ABCDE

User B

ABCD

CRDT Syncing

User A

ABCD

Insert E after D

$\{(A,(2)),\ (B,(4)),\ (C,(4,0)),\ (D,(4,1))\} \vee \{(E,(4,2))\}$

ABCDE

User B

ABCD

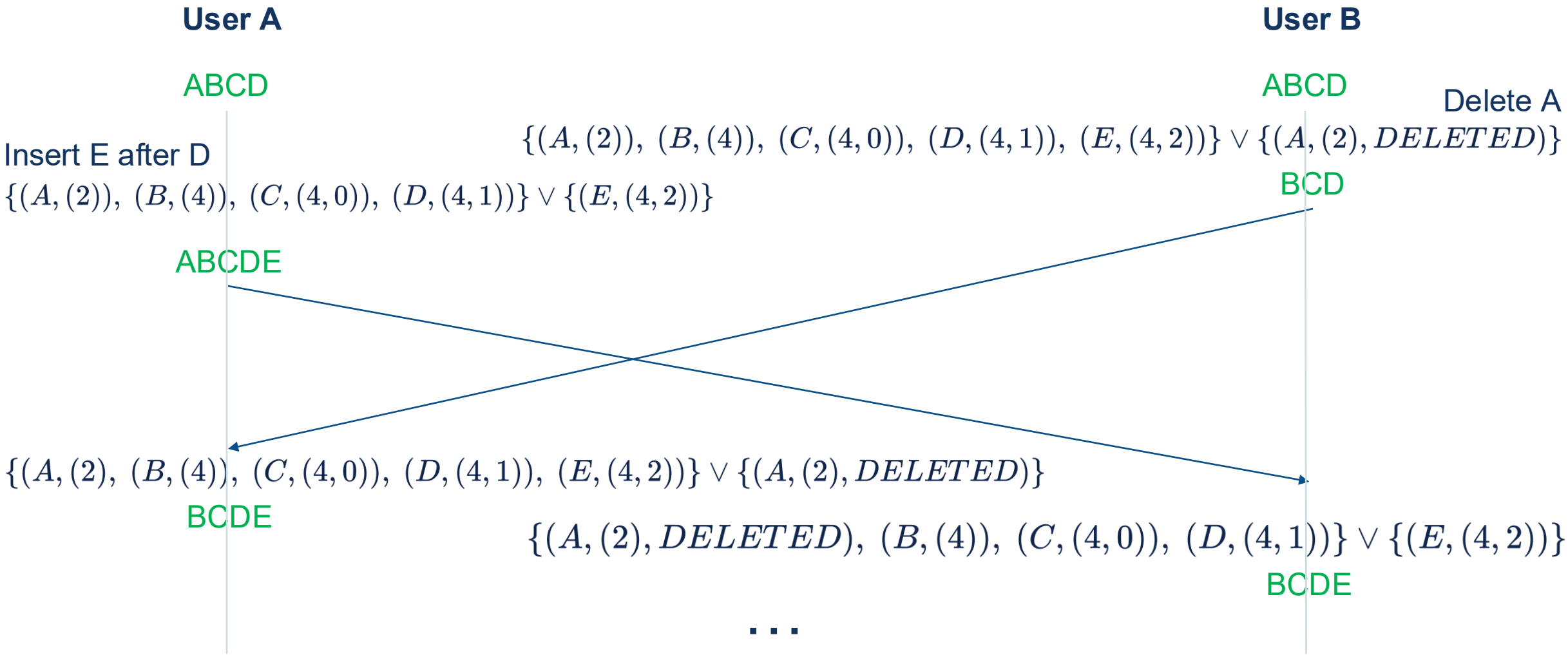
Delete A

$\{(A,(2)),\ (B,(4)),\ (C,(4,0)),\ (D,(4,1)),\ (E,(4,2))\} \vee \{(A,(2),DELETED)\}$

BCD

Note: Vector Clock is needed to detect which version is newer

CRDT Syncing



Interleaving problems with CRDT/LSEQ

Initial Document: Hi !

(1,0) → 'H' (1,1) → 'i' (1,2) → ' ' (9,0) → '!'

Interleaving problems with CRDT/LSEQ

Initial Document: Hi !

(1,0) → 'H' (1,1) → 'i' (1,2) → ' ' (9,0) → '!'

User 1 inserts: "students"

(2,0)→s, (3,0)→t, (4,0)→u, (5,0)→d, (6,0)→e, (7,0)→n, (8,0)→t, (8,1)→s

Interleaving problems with CRDT/LSEQ

Initial Document: Hi !

(1,0) → 'H' (1,1) → 'i' (1,2) → ' ' (9,0) → '!'

User 1 inserts: "students"

(2,0)→s, (3,0)→t, (4,0)→u, (5,0)→d, (6,0)→e, (7,0)→n, (8,0)→t, (8,1)→s

User 2 inserts: "friends"

(2,1)→f, (3,1)→r, (4,1)→i, (5,1)→e, (6,1)→n, (7,1)→d, (8,2)→s

Interleaving problems with CRDT/LSEQ

Initial Document: Hi !

(1,0) → 'H' (1,1) → 'i' (1,2) → ' ' (9,0) → '!'

User 1 inserts: "students"

(2,0)→s, (3,0)→t, (4,0)→u, (5,0)→d, (6,0)→e, (7,0)→n, (8,0)→t, (8,1)→s

User 2 inserts: "friends"

(2,1)→f, (3,1)→r, (4,1)→i, (5,1)→e, (6,1)→n, (7,1)→d, (8,2)→s

Synced Document: Hi s f t r u i d e n e d n t s s !

Interleaving and Jupiter OT

Interleaving can also occur in Jupiter OT

- User A types **"hello”,** User B types **"world"** at the same position, Result might be **"hweolrllod"** depending on operation timing and transformation

Jupiter transforms operations to preserve intent, if you meant to insert after "hello", the transformation tries to maintain that relationship

- LSEQ assigns permanent positions that don't change, which can lead to more unpredictable interleaving

Mitigations

- **Operational grouping**: Buffer rapid keystrokes into larger operations, batching rapid character insertions into word-level operations
- **Smart transformations**: Transformation functions that understand text structure
- **User feedback**: Real-time cursors and typing indicators to show concurrent editing
- **Conflict visualization**: Highlighting areas where concurrent edits occurred

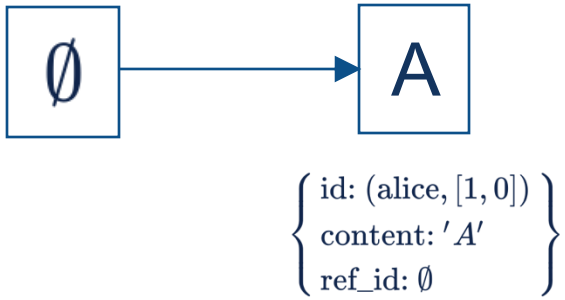
RGA (Replicated Growable Array)

$$\text{Character} = \left\{ \begin{array}{l} \text{id} : \text{SiteID} \times \text{VectorClock}, \\ \text{content} : \Sigma, \\ \text{ref_id} : \text{SiteID} \times \text{VectorClock} \cup \{\emptyset\} \end{array} \right\}$$



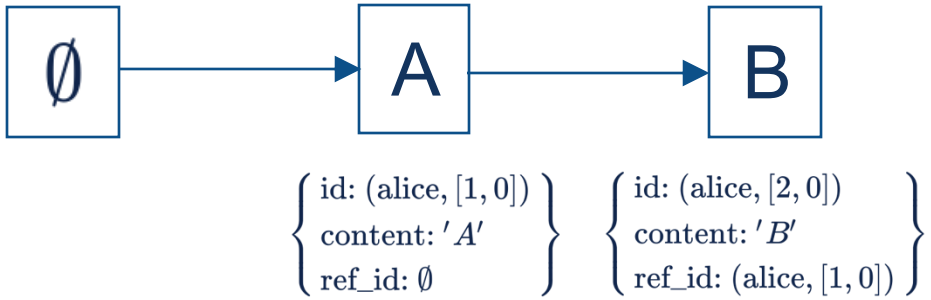
RGA (Replicated Growable Array)

$$\text{Character} = \left\{ \begin{array}{l} \text{id} : \text{SiteID} \times \text{VectorClock}, \\ \text{content} : \Sigma, \\ \text{ref_id} : \text{SiteID} \times \text{VectorClock} \cup \{\emptyset\} \end{array} \right\}$$



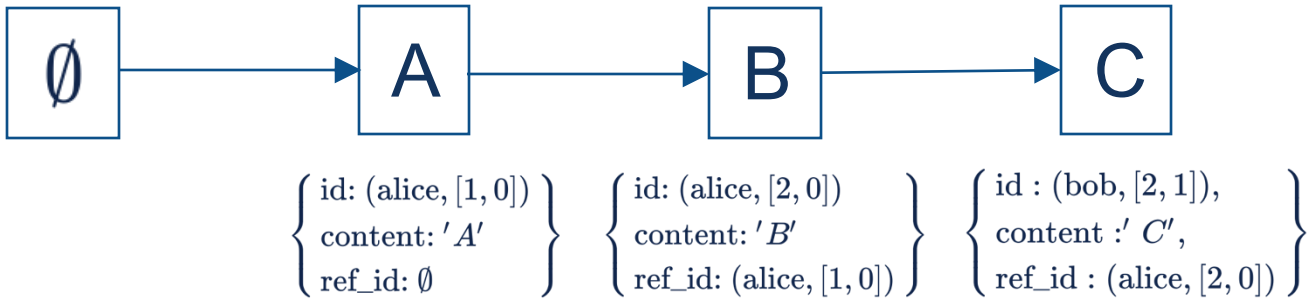
RGA (Replicated Growable Array)

$$\text{Character} = \left\{ \begin{array}{l} \text{id} : \text{SiteID} \times \text{VectorClock}, \\ \text{content} : \Sigma, \\ \text{ref_id} : \text{SiteID} \times \text{VectorClock} \cup \{\emptyset\} \end{array} \right\}$$



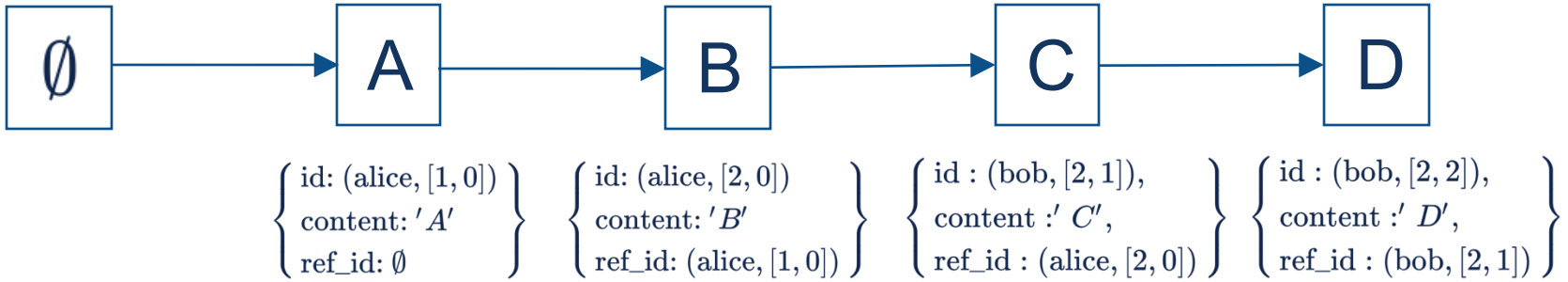
RGA (Replicated Growable Array)

$$\text{Character} = \left\{ \begin{array}{l} \text{id} : \text{SiteID} \times \text{VectorClock}, \\ \text{content} : \Sigma, \\ \text{ref_id} : \text{SiteID} \times \text{VectorClock} \cup \{\emptyset\} \end{array} \right\}$$



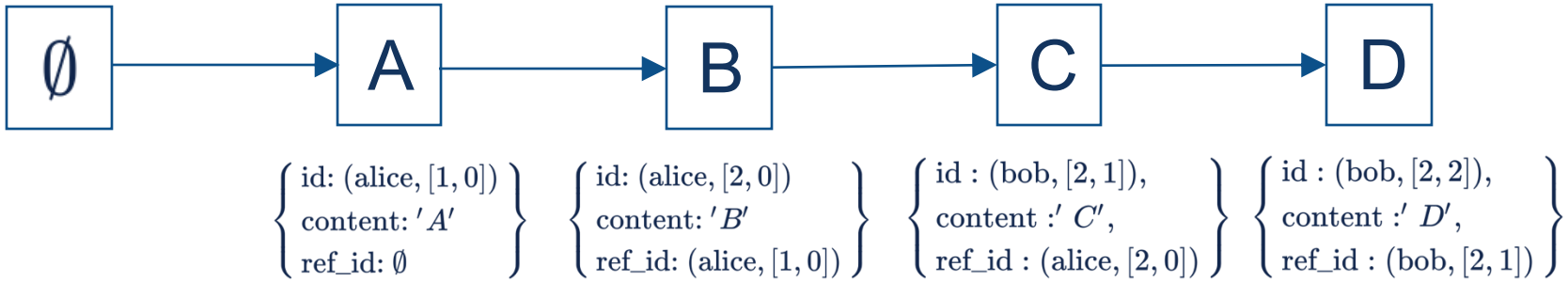
RGA (Replicated Growable Array)

$$\text{Character} = \left\{ \begin{array}{l} \text{id} : \text{SiteID} \times \text{VectorClock}, \\ \text{content} : \Sigma, \\ \text{ref_id} : \text{SiteID} \times \text{VectorClock} \cup \{\emptyset\} \end{array} \right\}$$



RGA (Replicated Growable Array)

$$\text{Character} = \left\{ \begin{array}{l} \text{id} : \text{SiteID} \times \text{VectorClock}, \\ \text{content} : \Sigma, \\ \text{ref_id} : \text{SiteID} \times \text{VectorClock} \cup \{\emptyset\} \end{array} \right\}$$

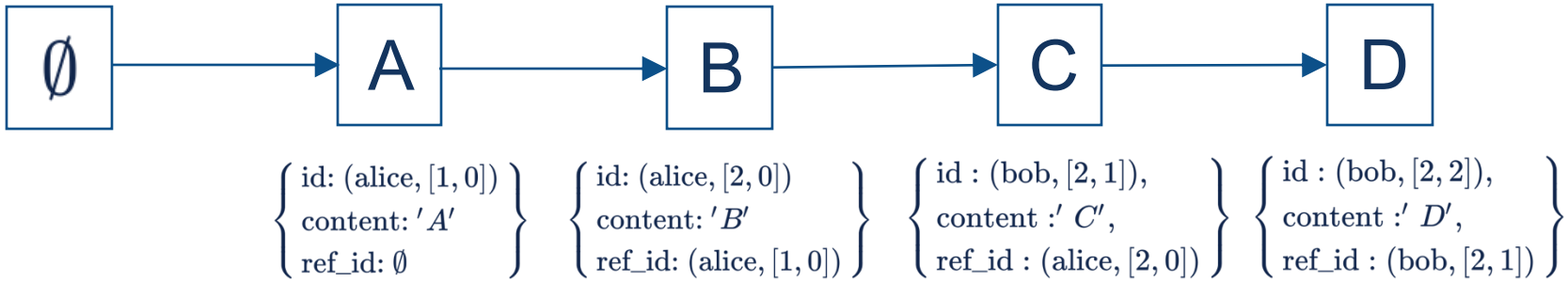


Alice inserts 'X' after 'b'

$$\text{Doc}_{Alice} = \text{Doc}_{Alice} \vee \left\{ \begin{array}{l} \text{id} : (\text{alice}, [3, 2]), \\ \text{content} :' X', \\ \text{ref_id} : (\text{alice}, [2, 0]) \end{array} \right\}$$

RGA (Replicated Growable Array)

$$\text{Character} = \left\{ \begin{array}{l} \text{id} : \text{SiteID} \times \text{VectorClock}, \\ \text{content} : \Sigma, \\ \text{ref_id} : \text{SiteID} \times \text{VectorClock} \cup \{\emptyset\} \end{array} \right\}$$



Alice inserts 'X' after 'b'

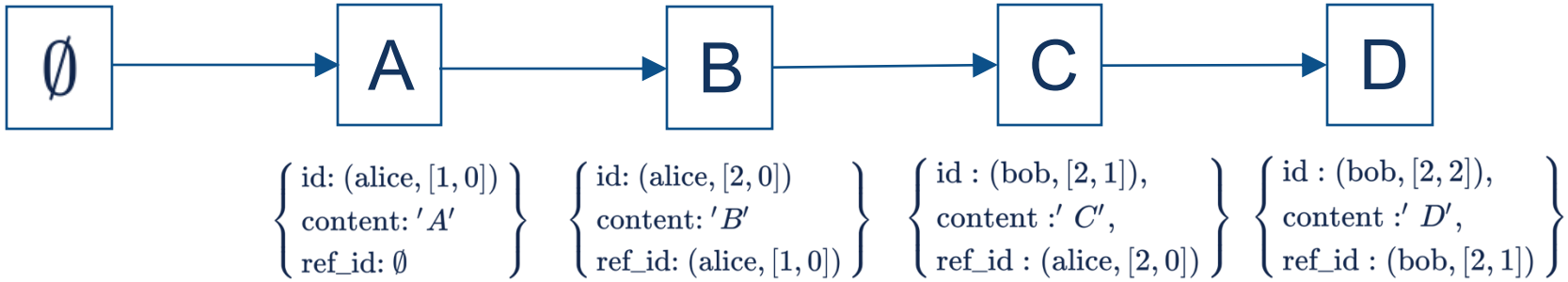
$$\text{Doc}_{Alice} = \text{Doc}_{Alice} \vee \left\{ \begin{array}{l} \text{id} : (\text{alice}, [3, 2]), \\ \text{content} :' X', \\ \text{ref_id} : (\text{alice}, [2, 0]) \end{array} \right\}$$

Bob (site=2): inserts 'Y' after ‘a'

$$\text{Doc}_{Bob} = \text{Doc}_{Bob} \vee \left\{ \begin{array}{l} \text{id} : (\text{bob}, [2, 3]), \\ \text{content} :' Y', \\ \text{ref_id} : (\text{alice}, [1, 0]) \end{array} \right\}$$

RGA (Replicated Growable Array)

$$\text{Character} = \left\{ \begin{array}{l} \text{id} : \text{SiteID} \times \text{VectorClock}, \\ \text{content} : \Sigma, \\ \text{ref_id} : \text{SiteID} \times \text{VectorClock} \cup \{\emptyset\} \end{array} \right\}$$

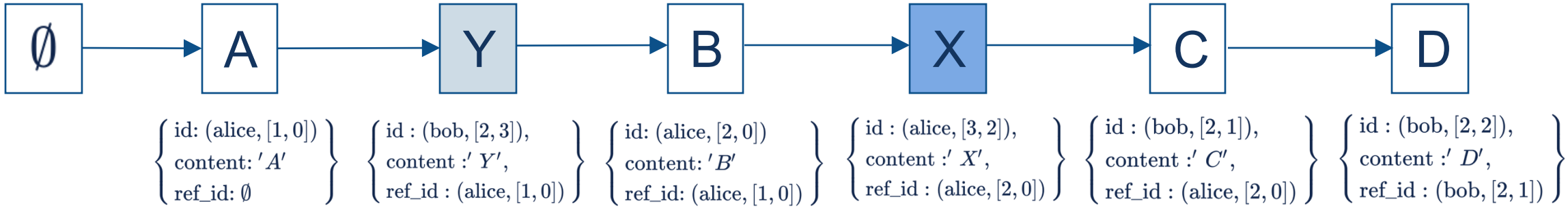


Alice inserts 'X' after 'b'

$$\text{Doc}_{Alice} = \text{Doc}_{Alice} \vee \left\{ \begin{array}{l} \text{id} : (\text{alice}, [3, 2]), \\ \text{content} :' X', \\ \text{ref_id} : (\text{alice}, [2, 0]) \end{array} \right\}$$

Bob (site=2): inserts 'Y' after ‘a'

$$\text{Doc}_{Bob} = \text{Doc}_{Bob} \vee \left\{ \begin{array}{l} \text{id} : (\text{bob}, [2, 3]), \\ \text{content} :' Y', \\ \text{ref_id} : (\text{alice}, [1, 0]) \end{array} \right\}$$



RGA Sorting

Reference position: space (user1, [3,0])

Children after sorting:

's' (user1, [5,0]) ← comes first after sorting
    't' (user1, [6,0])
        'u' (user1, [7,0])
            ... (rest of "students")
'f' (user2, [3,1]) ← comes second after sorting
    'r' (user2, [3,2])
        ... (rest of "friends")

Space (user1, [3,0])
→ 's' (user1, [5,0]) User 1 chain starts here
→ ‘f’ (user2, [3,1]) User 2 chain starts here

Since both "students" and "friends" reference the same position (after space), RGA uses ID tie-braker for ordering:

Final Document: "Hi students friends !"

Interleaving is still a problem in RGA, but it's significantly reduced compared to position-based CRDTs (LSEQ)

However, characters can be deleted by marking them as invisible (tombstoning). The document structure keeps growing because tombstones are retained to preserve reference relationships, though they can be garbage collected when safe

Operational Transformation

Pros

- **Strong intention preservation:** OT explicitly transforms operations to maintain the original intent of each edit, even when operations are applied in different orders
- **Compact representation:** Operations are typically small and efficient to transmit
- **Well-established** with proven implementations in systems like Google Docs
- **Fine-grained control:** Allows for sophisticated conflict resolution strategies

Cons

- **Requires central coordination:** Needs a server to order operations and handle transformations
- **Complex implementation:** Transformation functions must be carefully designed to maintain consistency
- **Scalability challenges:** Central server can become a bottleneck
- **Network dependency:** Offline editing is more difficult to implement

CRDT/LSEQ/RGA

Pros

- **Decentralised:** No central server required - peers can collaborate directly
- **Offline-friendly:** Works well in distributed, occasionally connected environments
- **Simpler reasoning:** Mathematical properties make correctness easier to verify

Cons

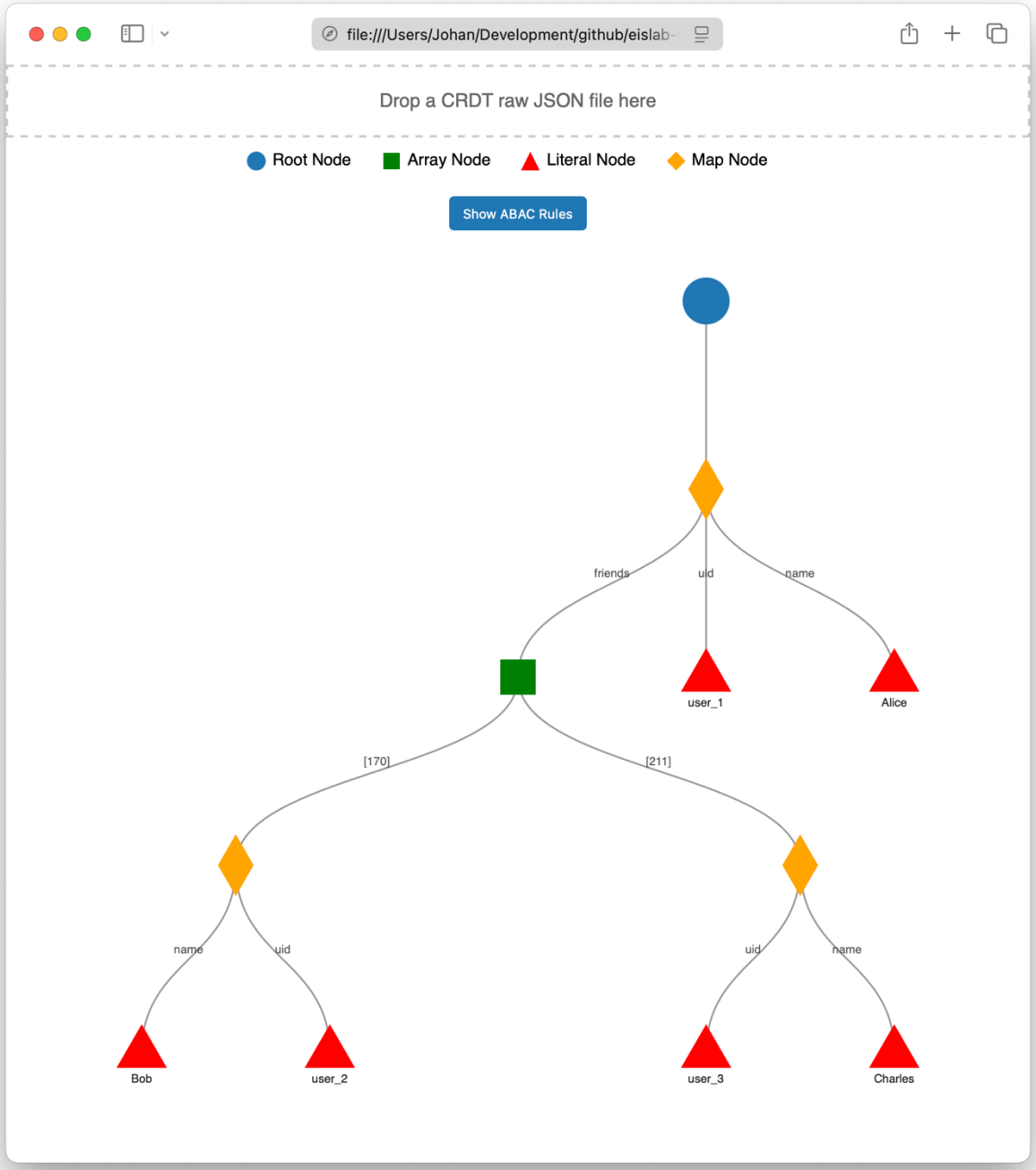
- **Weaker intention preservation:** May not preserve the original semantic intent of operations
- **Memory overhead:** Requires storing additional metadata (like unique identifiers for each character)
- **Interleaving:** Characters from different users can be inserted between intended sequences
- **Tombstone accumulation:** Deleted items often need to be retained for consistency

Research at LTU

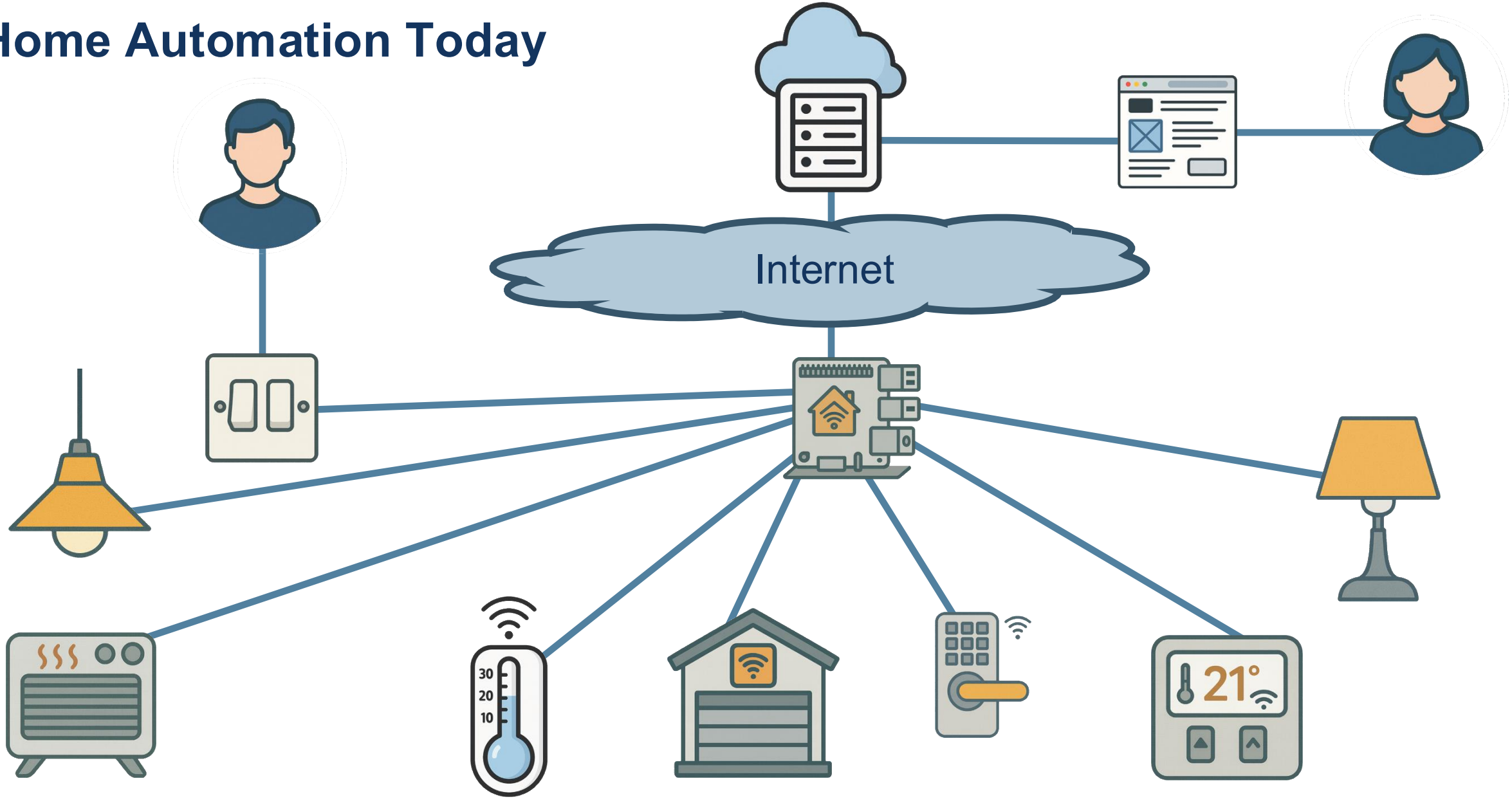
Cyber-Physical System Research group at LTU is exploring how to use CRDT to build *next-generation industrial systems* and *decentralized cloud systems*

- LWW + LSEQ
- JSON import/export
- Elliptic curve crypto
- Access control

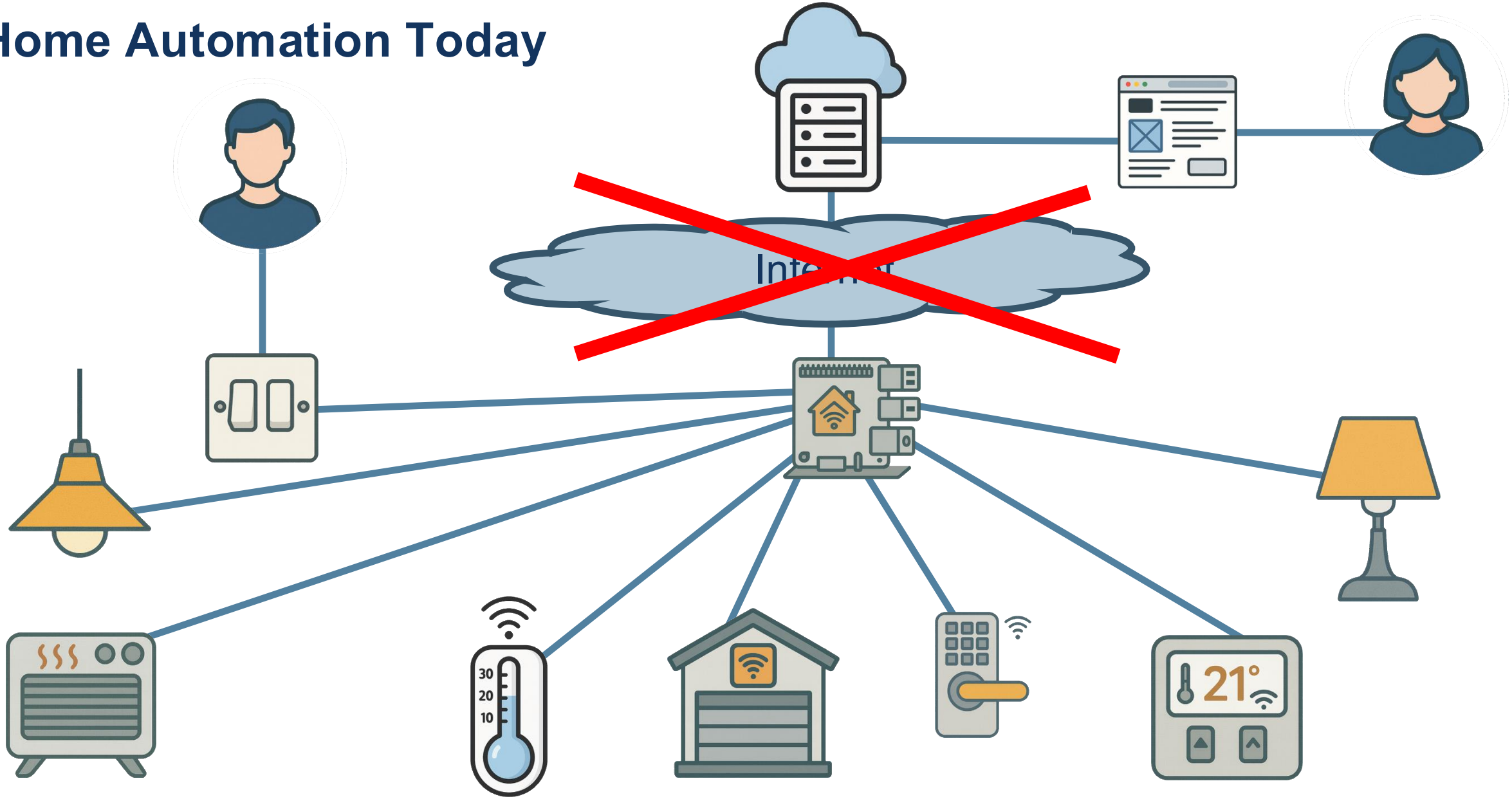
https://github.com/eislab-cps/synctree



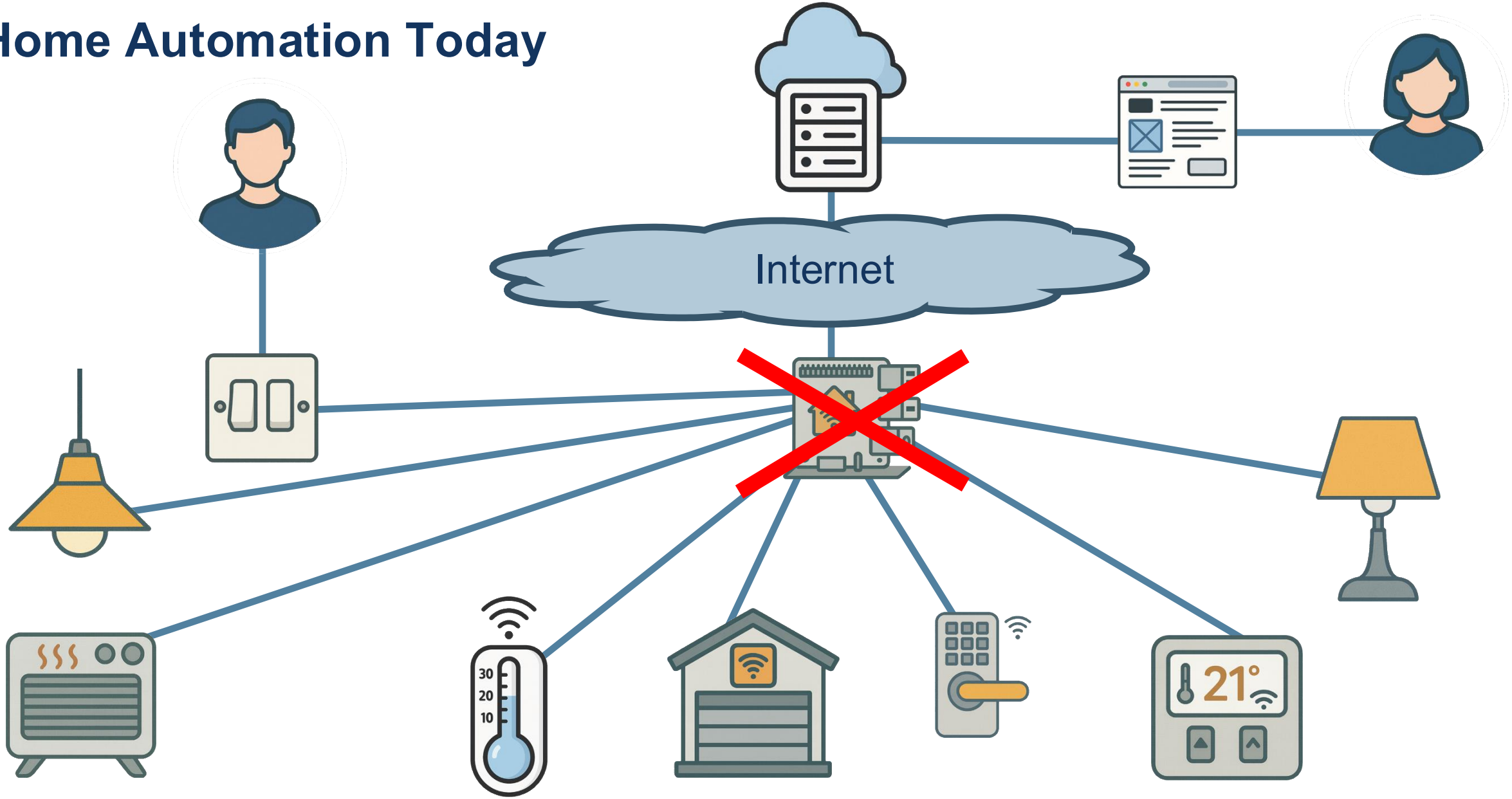
Home Automation Today



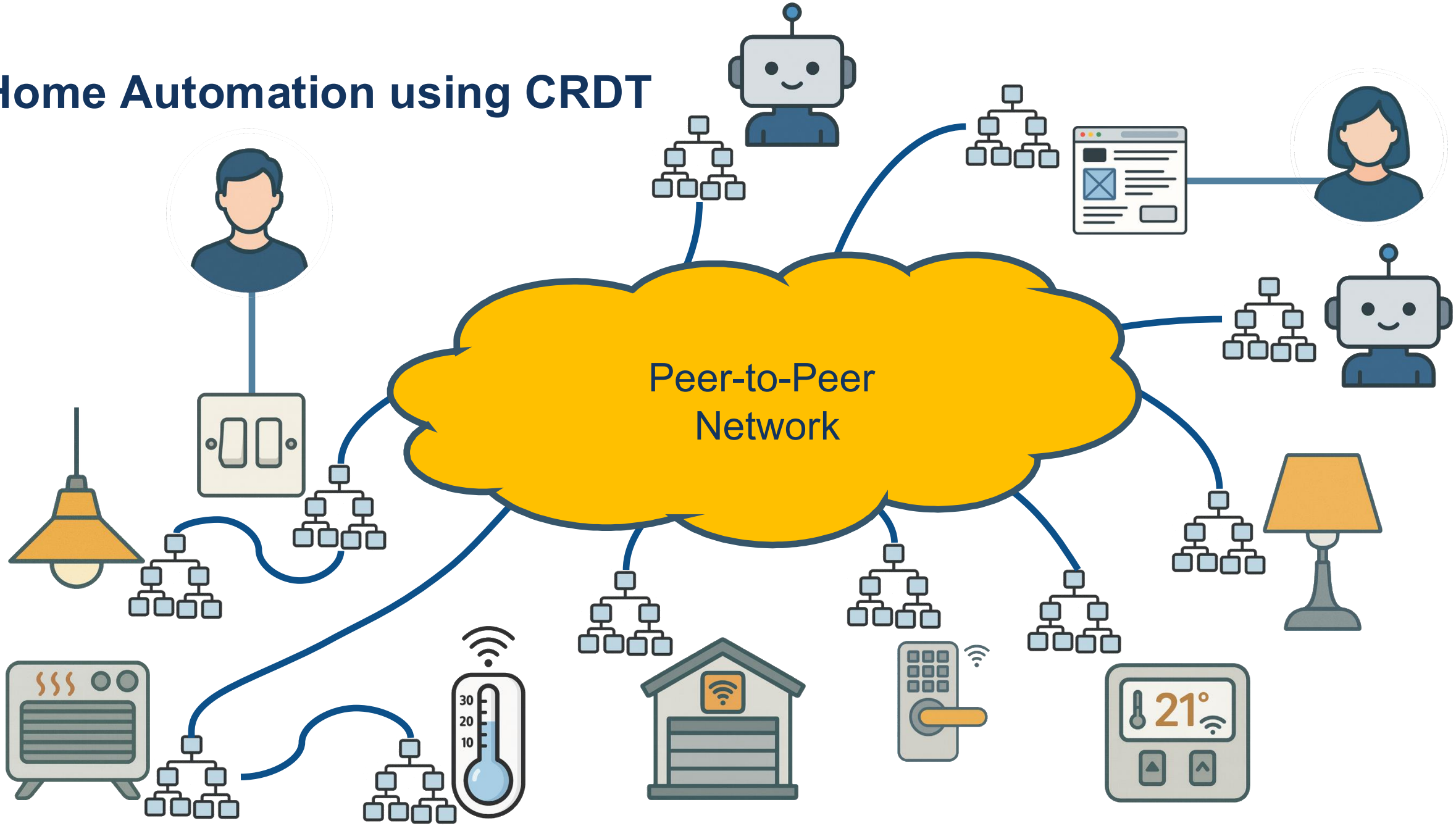
Home Automation Today



Home Automation Today



Home Automation using CRDT

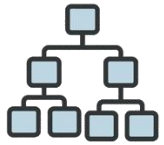


Cyber-Physical Reconciliation

CRDT serves as the digital twin by providing the distributed state management layer that keeps the digital replica synchronized across all participants in the system

- Complete history of state changes
- Offline Resilience
- Multiple simultaneous controllers

Agentic AI systems that can perceive, reason, and act autonomously by connecting to CRDT-based digital twins




```
{
  "lights": {
    "living_room": {"power": true, "brightness": 75},
    "bedroom": {"power": false, "brightness": 50},
    "kitchen": {"power": true, "brightness": 90}
  },
  "climate": {
    "thermostat": {"temp": 21, "mode": "heat"},
    "humidity": {"level": 45}
  },
  "security": {
    "garage_door": {"status": "closed", "locked": true},
    "front_door": {"locked": true},
    "alarm": {"armed": false}
  },
  "appliances": {
    "washing_machine": {"status": "idle", "cycle": "normal"},
    "robot_vacuum": {"status": "docked", "battery": 95}
  },
  "sensors": {
    "temperature": {"value": 20.5},
    "motion_living": {"detected": false},
    "motion_kitchen": {"detected": true}
  }
}
```

Readling list

Operational Transformation

High-latency, low-bandwidth windowing in the Jupiter collaboration system

https://dl.acm.org/doi/10.1145/215585.215706

CRDT

How CRDTs make multiplayer text editing part of Zed's DNA

https://zed.dev/blog/crdts

Logoot: A Scalable Optimistic Replication Algorithm for Collaborative Editing on P2P Networks

https://inria.hal.science/inria-00432368/document

LSEQ: an Adaptive Structure for Sequences in Distributed Collaborative Editing

https://hal.science/hal-00921633v1/document

An efficient collaborative editing algorithm supporting string-based operations (RGA)

https://ieeexplore.ieee.org/document/7565961

Differential Synchronization (Git-like syncing with Fussy Merge)

https://dl.acm.org/doi/10.1145/1600193.1600198


LULEÅ
UNIVERSITY
OF TECHNOLOGY